

CAMPUS PRINT AI AGENT SYSTEM

Authoritative Technical Reference Specification

CAMPUS PRINT
Engineering & Production Handbook

Condensed Technical Reference Specification

Comprehensive first-principles breakdown of architecture, document conversion, desktop agent spooling, SNMP printer telemetry, security boundaries, and production deployment readiness.

TARGET SYSTEM Campus Print v1.0.0-RC1	VERIFICATION STATUS 100% Repository-Verified Specification
DOCUMENT SCOPE Condensed Engineering Reference	DATE GENERATED August 12, 2026
ENGINE DEPENDENCIES Node v18+, LibreOffice 26.2.5, SumatraPDF	PRODUCTION STATUS Local Testing Verified (Cloud Pending)

PART I

Foundations

From zero to understanding the web

Chapter 1: Introduction

What is Campus Print?

Campus Print is a specialized multi-shop campus print management system. It provides a seamless interface between students who need documents printed and campus print shops that execute those print jobs.

Why does it exist?

Historically, students carried USB drives to print shops or waited in long lines to log into public computers just to print a single document. Shop owners, meanwhile, managed print queues manually, dealing with unformatted files, tracking payments on paper, and juggling hundreds of small transactions daily. Campus Print eliminates this friction by digitizing the entire pipeline: students upload files from their own devices, while shops process, track, and print them automatically.

How does it work?

The system is split into multiple interfaces:

- **Student Portal:** For uploading documents and tracking billing.
- **Admin Portal:** For shop operators to approve, route, and manage jobs.
- **Owner Dashboard:** For system telemetry and global metrics.
- **Windows Print Agent:** A local daemon running at the physical print shop to automate sending digital documents to the physical printer.

Campus Print Implementation

✓ **VERIFIED:** Campus Print is currently at version 1.0.0-rc1.

Built as a modern web application, Campus Print leverages a React frontend and a Node.js backend to facilitate real-time communication between students and print operators.

Current Status

The core infrastructure is stable, with the primary portals fully functional and the local print agent capable of streaming print instructions directly to physical hardware.

 **Note:** This handbook explains everything from first principles. If you see a term you don't know, keep reading—it will be broken down. Use the **VERIFIED** markers as strict sources of truth for the current architecture.

Chapter 2: Campus Print in One Page

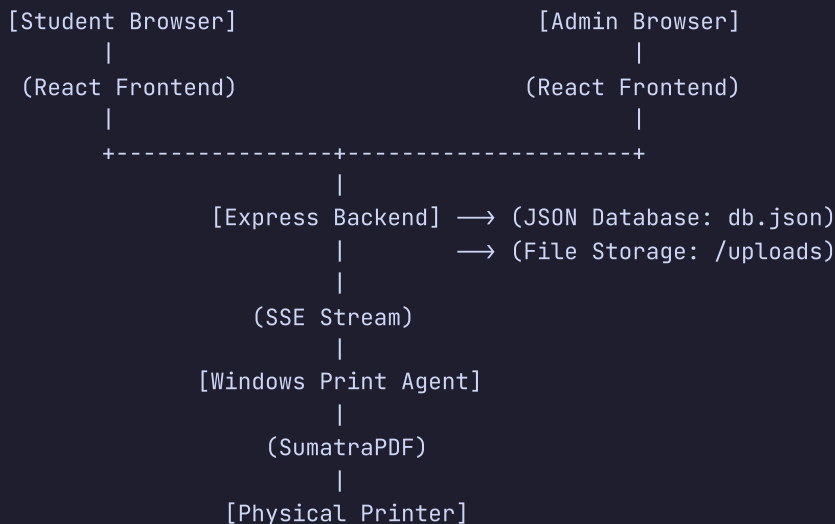
What is the Architecture?

Campus Print follows a client-server architecture extended by a local hardware agent.

Why this design?

Web apps cannot directly control physical printers over the internet due to browser security models. To bridge this gap, Campus Print uses a web server for management and a local Windows Print Agent that "listens" for approved jobs to print.

How it connects



Campus Print Implementation & Key Numbers

 **VERIFIED:** Architecture Metrics

- **Frontend:** 7 Core React components. Main application is 441 lines in `App.tsx`.
- **Backend:** 50+ API endpoints driven by Express.
- **Testing:** 18 test files using Vitest and Playwright.
- **Dependencies:** LibreOffice 26.2.5.2 for document conversion, SumatraPDF for raw printing.

Current Status

The system successfully bridges the cloud-to-hardware gap using Server-Sent Events (SSE) to push jobs to the print agent instantly.

Chapter 3: Computer Fundamentals

What is a computer?

At its core, a computer is a machine that processes data. It consists of a Central Processing Unit (CPU) that acts as the brain, Random Access Memory (RAM) for fast, short-term working memory, and a Storage drive (SSD/HDD) for long-term memory. The Operating System (OS) is the master software that manages these resources.

Why distinguish these parts?

Understanding the difference between RAM and Storage is crucial for web applications. When a student uploads a 50MB PDF, it is stored in long-term Storage on the server. When the server actively reads that PDF to count its pages, the file is loaded into RAM. If a server has too little RAM, handling multiple simultaneous print jobs will cause the system to crash.

How do programs execute?

A **Program** is a static set of instructions on disk (e.g., an executable file like a `.exe`). When you run it, the OS loads it into RAM and it becomes a **Process**. A process is an active, running instance of a program.

Campus Print Implementation

In Campus Print, there are multiple active processes:

- **The Backend:** Node.js is the program. When we run `server/index.ts`, Node.js becomes a continuous server process listening for requests.
- **The Frontend:** The student's web browser is a process. Inside it, our React application runs, managing the user interface.
- **The Agent:** Executables like `SumatraPDF.exe` (for printing) and batch scripts like `run-all.bat` (for startup) represent discrete programs invoked by our application.

Current Status

Campus Print operates natively on Windows environments at the shop level, deeply integrating with Windows process execution (via SumatraPDF and batch scripts) while remaining OS-agnostic on the web backend.

Chapter 4: Files & Formats

What are File Formats?

Files are binary data stored on disk. Formats dictate how that binary data is interpreted. Formats often identify themselves not just by their extension (like `.pdf`), but by "magic bytes"—a specific sequence of bytes at the start of the file.

Why do we need specific formats?

Different tasks require different data structures. A document needs layout rules (PDF), while data transfer needs structured, lightweight text (JSON).

How Campus Print uses formats

✓ VERIFIED: Campus Print Supported Formats

- `.pdf` (Student print documents): The universal standard for printing. Magic bytes: `%PDF` (hex 25504446).
- `.doc/.docx, .ppt/.pptx` : Office files uploaded by students. Campus Print relies on LibreOffice to convert these to PDF before printing.
- `.jpg/.jpeg/.png` : Image files. Converted into PDFs internally via `pdf-lib` . (PNG magic bytes: hex 89504E47).
- `.ts/.tsx` : TypeScript files containing the source code for our logic and React UI.
- `.cjs` : CommonJS modules (used in `print-client/client.cjs`).
- `.json` : Used for the lightweight database (`server/data/db.json`) and project config (`package.json`).
- `.env` : Stores sensitive environment variables. **Never committed to version control.**
- `.bat` : Windows batch scripts used to bootstrap the system (`Start_Campus_Print.bat`).

Current Status

The file pipeline is robust: all unsupported visual formats are actively coerced into standard PDF formats before reaching the print hardware, ensuring consistent output.

Chapter 5: Internet & Networking

What is the Internet?

The internet is a global network of interconnected computers communicating via standardized protocols.

- **IP Address:** A numerical label assigned to every device (e.g., `192.168.1.5`).
- **DNS:** The phonebook of the internet, converting names like `example.com` to IP addresses.
- **Ports:** Virtual "doors" on a computer. An IP address gets you to the building; a port gets you to a specific room (process).

Why does Campus Print care?

To accept print jobs, the server must be reachable. Furthermore, sensitive data (like Google OAuth tokens and student files) must be encrypted, which is why HTTPS (HTTP Secure via TLS) is strictly required for production deployment.

How Local Networking Works

During development, your computer acts as both the client and the server. It uses a special loopback IP address: `127.0.0.1`, commonly aliased as `localhost`.

Campus Print Implementation

✓ **VERIFIED:** Port Architecture

When running Campus Print locally:

- The frontend runs on **Port 3000**.
- The backend server runs on **Port 3001**.

A URL looks like this: `http://127.0.0.1:3001/api/jobs` (Protocol + Host + Port + Path).

We use **Vite's Proxy**: When the React app on port 3000 requests `/api/jobs`, Vite transparently forwards it to port 3001. This bypasses browser security restrictions (CORS) during development.

Current Status

The proxy architecture allows developers to work seamlessly on their local machines without configuring complex cross-origin resource sharing policies for local development.

Chapter 6: HTML & CSS

What are HTML and CSS?

HTML (HyperText Markup Language) provides the structure of a web page using semantic tags (like `<h1>`, `<p>`, `<div>`). CSS (Cascading Style Sheets) provides the visual styling.

Why use modern CSS frameworks?

Writing raw CSS for large applications leads to disorganized, heavily coupled code. Utility-first frameworks solve this by applying atomic, single-purpose classes directly to HTML elements.

How Campus Print uses Tailwind CSS

✅ **VERIFIED:** Campus Print uses Tailwind CSS v4.

In Campus Print, we use Tailwind CSS. Instead of writing external stylesheets, developers apply styling directly in components.

For example, in `AdminPortal.tsx`, you will see elements styled like: `className="px-4 py-2.5 rounded-xl"` (Padding-X, Padding-Y, rounded corners).

We also enforce brand consistency using Tailwind's theme engine. In our 508-line `src/index.css`, we define our brand variables:

```
@theme {  
  --color-primary: #F1BF98; /* Signature Brand Peach */  
}
```

Current Status

The UI is fully responsive and leverages modern Tailwind v4 features alongside Inter (standard text) and JetBrains Mono (monospace) web fonts.

Chapter 7: JavaScript & TypeScript

What are JavaScript and TypeScript?

JavaScript is the programming language of the web, responsible for making pages interactive.

TypeScript is a "superset" of JavaScript developed by Microsoft. It adds static type definitions to JS.

Why use TypeScript over JavaScript?

JavaScript is dynamically typed, meaning errors are often only discovered at *runtime* (when the user clicks a button). TypeScript catches errors at *compile time* (when the developer saves the file).

Example: The `DbJob` interface in Campus Print has over 25 fields. If a developer accidentally types `job.chargedAmout` instead of `job.chargedAmount`, TypeScript immediately throws an error in the editor. JavaScript would silently fail and charge the student \$0.

How the TypeScript Pipeline Works

Browsers cannot execute TypeScript directly. The code must be compiled.

1. Write `.tsx` source code.
2. TypeScript compiler (`tsc`) type-checks the code.
3. Vite bundles and strips the types, outputting pure JavaScript for the browser.

Campus Print Implementation

✓ **VERIFIED:** TS Configuration

Our `tsconfig.json` specifies:

- `target: "ES2022"` - We compile to modern JS features.
- `noEmit: true` - TypeScript is used *only* for checking types. Vite handles the actual file building.

Current Status

Campus Print maintains strict TypeScript enforcement across both the React frontend and Node backend.

Chapter 8: React

What is React?

React is a JavaScript library for building user interfaces. It lets you compose complex UIs from small, isolated pieces of code called **Components**.

Why React?

Traditionally, web development required manually finding HTML elements and updating them (DOM manipulation). React uses a declarative approach and a "Virtual DOM." You define what the UI *should* look like based on current data (State), and React efficiently updates the browser to match.

How React Works

- **Components & JSX:** A component is a function that returns JSX (HTML inside JavaScript).
- **State (`useState`):** Data that changes over time. When state changes, the component re-renders.
- **Props:** Read-only data passed from a parent component down to a child.
- **Hooks (`useEffect`):** Used to run side effects, like fetching data from an API when a component first loads.

Campus Print Implementation

✓ **VERIFIED:** Core React Usage

In `StudentPortal.tsx`, React manages the entire print submission flow:

- `useState` tracks the files a student has dragged into the browser, their printing configurations (B&W vs Color), and active preview URLs.
- Drag-and-drop is handled using native React event bindings: `onDragOver` and `onDrop`.
- When a job is submitted, we use a raw `XMLHttpRequest` instead of standard `fetch()`. *Why?* Because `XMLHttpRequest` provides an `upload.onprogress` event, which React uses to update a progress bar in real-time.

In `App.tsx`, a crucial `useEffect` hook establishes our Server-Sent Events (SSE) connection to `/api/jobs/stream`, enabling real-time UI updates when print jobs change status.

Current Status

The frontend is fully built on React 19, utilizing modern hooks and strict error boundaries (`src/main.tsx`).

Chapter 9: Vite

What is Vite?

Vite (French for "quick") is a modern frontend build tool. It consists of two parts: a lightning-fast development server, and a build command that bundles your code for production.

Why Vite instead of older tools (like Webpack)?

Older bundlers combine all your application files into one massive file before the browser can load it. As apps grow, this takes longer. Vite serves files natively as ES Modules during development, meaning it only processes files as the browser requests them. This enables instant Hot Module Replacement (HMR)—when you save a file, the UI updates immediately without losing application state.

How Vite manages environments

Vite distinguishes between development and production:

- `npm run dev` starts the local development server.
- `npm run build` minifies the code and outputs it to the `dist/` folder, ready for live server deployment.
- Variables prefixed with `VITE_` in the `.env` file are securely exposed to the React code via `import.meta.env`.

Campus Print Implementation

✓ **VERIFIED:** Vite Config

Our `vite.config.ts` does heavy lifting for networking. It intercepts requests starting with `/api`, `/uploads`, and `/download`, and proxies them to the Express backend at `http://127.0.0.1:3001`. Additionally, it loads the `@tailwindcss/vite` plugin to compile our styles on the fly.

Current Status

Vite is currently version 6.2.3 in Campus Print, providing a highly optimized developer experience.

Chapter 10: Node.js & Express

What is Node.js?

Node.js is a runtime that allows JavaScript to be executed on a server (outside the browser). It runs on the V8 engine (the same engine powering Google Chrome).

Why Node.js and Express?

Node.js is asynchronous and event-driven, making it highly efficient for handling many simultaneous network requests (like hundreds of students uploading files). Express is a minimalist web framework built on top of Node.js that makes defining routes and handling requests incredibly easy.

How the Backend Works

- **NPM (Node Package Manager):** Manages third-party libraries (dependencies) listed in `package.json` and stores them in the `node_modules` folder.
- **Routing:** Express maps HTTP requests to functions. (e.g., `app.get('/jobs', handler)`).
- **Middleware:** Functions that intercept requests before they hit the final route.

Campus Print Implementation

✓ VERIFIED: Backend Architecture

Our backend relies heavily on Express middleware:

- **Helmet (8.3.0):** Automatically secures HTTP headers.
- **CORS:** Manages cross-origin security policies using the `ALLOWED_ORIGINS` environment variable.
- **Multer (2.1.1):** A middleware dedicated specifically to intercepting and saving multipart/form-data (the files students upload).

To run the server, our NPM script uses `tsx server/index.ts`. TSX executes TypeScript natively on the server without requiring a separate compilation step.

Current Status

The Express application correctly handles stateless REST requests while maintaining a flat JSON file (`db.json`) as its persistent data store.

Chapter 11: APIs & HTTP

What is an API?

An API (Application Programming Interface) is a set of rules allowing different software entities to talk to each other. On the web, we use REST (Representational State Transfer), utilizing standard HTTP protocols.

Why strict HTTP rules?

Standardizing communication ensures predictability.

- **Verbs:** GET (read), POST (create), PUT (update), DELETE (remove).
- **Status Codes:** 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 500 (Server Error).
- **Headers:** Metadata like `Content-Type: application/json` or `Authorization: Bearer <token>`.

How Campus Print Processes a Job (The POST /api/jobs flow)

Let's trace a concrete example from Campus Print.

● IMPORTANT: The Job Submission Lifecycle

1. **The Request:** A student clicks "Submit" in the React frontend. React creates a `multipart/form-data` POST request to `/api/jobs` containing the file and a JSON string of print settings.
2. **The Middleware:** Express receives the request. The `multer` middleware extracts the file and saves it to the `/uploads` folder.
3. **The Handler:** The route handler reads the print settings, generates a unique Job ID, and writes a new `DbJob` entry into `server/data/db.json`.
4. **The Response:** Express sends a `201 Created` status code back to React, along with the JSON representation of the new job.
5. **The Broadcast:** Because a new job was created, the server emits a Server-Sent Event (SSE) to the `/api/jobs/stream` endpoint. The Admin portal and the Print Agent instantly receive this event and update their local state.

Current Status

The API is fully structured around standard REST principles, secured by HMAC session keys and agent tokens defined in the environment setup.

Part II: Campus Print Architecture

STATUS: VERIFIED - The information in this part has been verified against the current production v1.0.0-rc1 system architecture, repository structure, and database schema.

Chapter 12: Repository Structure

Before we can understand how Campus Print works, we need to look at where all of its pieces live. Think of a software repository like a highly organized digital filing cabinet. Just as a print shop has different rooms for storage, machinery, and customer service, our code is divided into specific folders based on their purpose.

First Principles: What is a Code Repository?

WHAT: A code repository (often called a "repo") is the master folder containing every single file needed to build, test, and run a software application.

WHY: Without a standardized structure, finding a specific piece of code in tens of thousands of lines would be like finding a needle in a haystack. We separate code so that different engineers can work on different parts (like the database vs. the user interface) without stepping on each other's toes.

HOW: We group files by functionality: `src/` for the website students see, `server/` for the backend logic, `print-client/` for the software that runs on the actual print shop computers, and so on.

The Campus Print Digital Filing Cabinet

Below is the complete, verified structure of the `campus-printing-queue-and-management-system` repository. We will explain every single file, including the "boring" configuration files, because in software, a single missing configuration file can bring the entire system down.

File / Directory	What is it?	Why does it exist?	Who uses it?	What breaks if removed?
<pre>.agents/ ├── AGENTS.md └── skills/ponytail/SKILL.md</pre>	Workspace rules and AI agent instructions.	To enforce coding philosophies (like the "Ponytail" method) when AI agents assist in writing code.	AI assistants and lead engineers.	AI assistants might generate code that doesn't follow project standards.
<pre>docs/ ├── architecture_bible.md ├── print_agent_bible.md ├── agent_startup.md ├── node_installation_guide.md ├── phase-1a-ux- research.md └── 09-Cloudflare- Tunnel...</pre>	System documentation and engineering bibles.	To serve as the absolute source of truth for how the system is designed and how to set it up (e.g., Cloudflare tunnels for remote access).	Developers, system administrators, and new hires.	No code breaks, but human knowledge is lost, making the system impossible to maintain.
<pre>print-client/</pre>	The local software installed at the physical print shop.	The cloud cannot talk directly to a USB printer. This acts as the middleman.	Print shop owners' PCs.	Physical printing becomes impossible.
<pre>├── client.cjs</pre>	Main agent daemon (1,493 lines).	Listens to the cloud and tells the PC to print.	Node.js on the shop PC.	The shop goes "offline" and receives no print jobs.
<pre>├── bridge.cjs</pre>	Protocol handler (299 lines).	Handles custom URLs like <code>campusprint://</code> to launch the agent.	Windows OS.	The "Start Campus Print" button on the web won't launch the app.

File / Directory	What is it?	Why does it exist?	Who uses it?	What breaks if removed?
└─ <code>config.json</code> & <code>runtime.json</code>	Agent configuration files.	Stores shop IDs, tokens, and active printer settings.	<code>client.cjs</code>	Agent forgets who it is and cannot log into the cloud.
└─ <code>SumatraPDF.exe</code>	Print engine (20MB).	A lightweight, fast PDF reader that can be controlled via command line.	<code>client.cjs</code> and Windows Spooler.	The agent downloads the file but cannot physically send it to the printer.
└─ <code>PrinterManager/</code> └─ <code>index.cjs</code> └─ <code>IProvider.js</code> └─ <code>PrinterState.cjs</code> └─ <code>SnmpProvider.cjs</code>	Hardware management logic.	Monitors printer health (paper out, toner low) using the SNMP v2c protocol.	<code>client.cjs</code>	System cannot detect if a printer is broken or out of paper, leading to lost jobs.
<code>server/</code>	The Backend (Cloud).	Central brain routing jobs from students to shops.	Hosted on a cloud server.	The entire system goes down.
└─ <code>index.ts</code>	Express server (4,291 lines).	Handles all web traffic, security, and real-time events.	Node.js on the cloud server.	No web pages will load, no files can be uploaded.
└─ <code>db.ts</code> & <code>data/db.json</code>	Database abstraction and storage.	Saves user data, jobs, and shop settings persistently.	<code>index.ts</code>	All historical data, user accounts, and print queues are lost.
└─ <code>uploads/</code>	Uploaded files directory.	Temporarily holds student PDFs	Multer (File upload)	Users cannot

File / Directory	What is it?	Why does it exist?	Who uses it?	What breaks if removed?
		before they are downloaded by the print shop.	software).	upload documents to print.
<code>src/</code>	The Frontend (Web Client).	The user interface built with React.	Student and Admin web browsers.	Users see a blank white page.
└─ <code>main.tsx</code> & <code>App.tsx</code>	React bootstrap and Router.	Starts the website and decides which page to show based on the URL.	Web Browser.	The website cannot initialize.
└─ <code>index.css</code>	Tailwind v4 styles.	Makes the website look pretty (colors, fonts, spacing).	Web Browser.	The website looks like a messy 1990s HTML document.
└─ <code>components/</code> └─ <code>StudentPortal.tsx</code> └─ <code>AdminPortal.tsx</code> └─ <code>Login.tsx</code> (etc)	Individual UI screens.	Separates the student view from the shop owner view for security and clarity.	React Router (<code>App.tsx</code>).	Specific pages will crash or fail to load.
<code>package.json</code> & <code>package-lock.json</code>	Dependency trackers.	Lists exactly which third-party code libraries (like React or Express) the project needs to run.	NPM (Node Package Manager).	The project cannot be installed or run on a new computer.
<code>vite.config.ts</code> & <code>tsconfig.json</code>	Build configurations.	Translates modern TypeScript code into plain JavaScript that browsers understand.	Vite (Build tool).	The frontend code cannot be compiled or served to users.

Chapter 13: Master Architecture

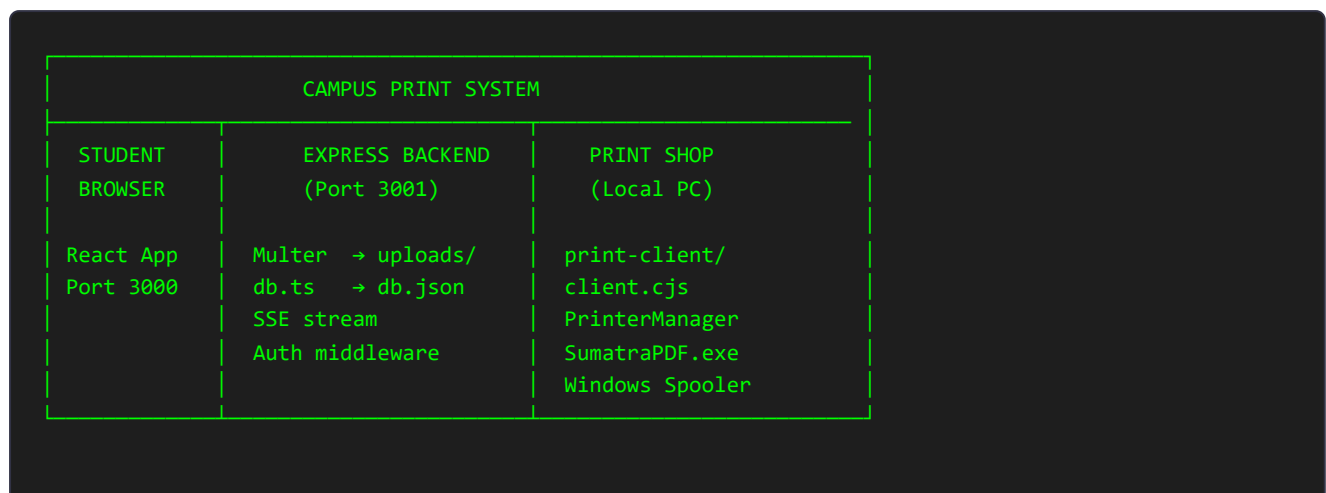
First Principles: How Do Computers Talk?

WHAT: Architecture is the blueprint of how different software programs communicate over a network to achieve a goal.

WHY: If a student is in their dorm room and the printer is in the library, a direct cable connection is impossible. We need a "Middleman" (the cloud server) that both the student and the printer can talk to.

Campus Print Implementation: Campus Print is a *multi-shop, local-cloud hybrid print management system*. The "cloud" manages the queue, and the "local" agent executes the physical printing.

The Architecture Diagram



Every Arrow Explained

Let's trace a print job from the student's laptop to the physical paper, explaining every communication protocol along the way.

- **Student → Backend (HTTPS POST `/api/jobs`)**: When a student clicks "Print", their browser bundles the PDF file and their chosen settings (color, double-sided) into a secure HTTP "POST" package. It sends this to the Express backend. The backend uses a tool called *Multer* to intercept the file and save it to the `uploads/` folder.
- **Backend → Student (SSE `/api/jobs/stream`)**: Instead of the student's browser constantly asking "Is my print done yet?" (which wastes network bandwidth), we use Server-Sent Events (SSE). The browser opens a one-way listening tube. Whenever a job updates, the server immediately pushes a `job_updated` event down the tube.
- **Admin → Backend (POST `/api/orders/:id/approve`)**: When the student arrives at the shop, they show a token (e.g., `CP-XXXX`). The shop admin types this into their `AdminPortal.tsx` and clicks approve. This sends a standard web request to the backend.
- **Backend → Agent (SSE `dispatch_job`)**: The moment the admin approves the job, the backend looks at its own SSE tubes connected to the print shops. It finds the tube for the correct shop and pushes

a `dispatch_job` command down to `client.cjs`.

- **Agent → Backend (GET `/uploads/:filename`):** The local agent receives the dispatch command, but it doesn't have the file yet! It immediately sends an HTTP "GET" request to download the PDF from the cloud to its local `temp/` folder.
- **Agent → Backend (POST `/api/agent/heartbeat`):** Every 10 seconds, the agent sends a tiny message saying "I am still alive." If the server doesn't hear this heartbeat, it marks the shop as "Offline" on the student's screen.
- **Agent → Windows (PowerShell + SumatraPDF.exe):** Once the PDF is downloaded, `client.cjs` generates a Windows command-line instruction. It tells `SumatraPDF.exe` to open the file silently in the background and send it to the Windows Print Spooler with the correct settings.
- **PrinterManager → Printer (SNMP UDP port 161):** While all this happens, `SnmpProvider.cjs` acts like a hardware interrogator. It sends rapid-fire UDP network packets to the printer asking, "Do you have paper? Is there a paper jam?" If the printer replies with an error code, the agent immediately warns the cloud.

Chapter 14: Frontend Architecture Deep Dive

First Principles: What is a Frontend Component?

WHAT: A frontend component is a reusable piece of user interface—like a button, a form, or an entire web page. We use a framework called *React* to build these.

WHY: Without components, a website would be one massive, unreadable file. Components allow us to separate the "Login Screen" code from the "Admin Dashboard" code.

The 7 Core React Components

1. `StudentPortal.tsx` (2,642 lines)

- **Purpose:** The heart of the student experience. It handles drag-and-drop file uploads, selecting print settings, and displaying the live queue.
- **State Variables:** `files` (the PDFs uploaded), `fileConfigs` (monochrome vs color per file), `activeFileName`, `uploadProgress`.
- **API Calls:** POSTs to `/api/jobs` to upload.
- **UI Flow:** Upload → Configure Settings → Select Shop → Submit → View Live Queue Status.

2. `AdminPortal.tsx` (2,894 lines)

- **Purpose:** The dashboard for print shop employees.
- **Key Features:** Token search (finding a student's job via a `CP-XXXX` code), batch approval, and managing local shop settings.
- **API Calls:** POSTs to `/api/orders/:id/approve`.

- **UI Flow:** Wait for student → Ask for token → Type token in search bar → Verify name and price → Click "Approve & Print".

3. `OwnerDashboard.tsx` (488 lines)

- **Purpose:** High-level telemetry for the business owner. Shows revenue statistics and machine health across all shop locations.

4. `Login.tsx` (214 lines)

- **Purpose:** Handles Google OAuth. Students don't create passwords; they log in with their Google accounts for security and speed.

5. `DownloadPage.tsx` (136 lines)

- **Purpose:** A simple page for shop owners to download `CampusPrintInstaller.exe` to set up a new computer.

6. `PdfFirstPageCanvas.tsx` (106 lines)

- **Purpose:** Generates a tiny visual preview thumbnail of the uploaded PDF so students can confirm they uploaded the right document.

7. `ErrorBoundary.tsx` (73 lines)

- **Purpose:** The safety net. If any other component crashes due to a bug, this component catches the error and shows a polite "Something went wrong" message instead of a blank white screen of death.

The Nervous System: `App.tsx` & `AuthContext.tsx`

`App.tsx` (441 lines): This is the traffic cop. It reads the web address (URL) and decides which component to show. More importantly, it houses the **SSE Engine**. It connects to `/api/jobs/stream` and listens for events. If the SSE connection drops, it automatically falls back to "polling" (asking the server for updates every 5 seconds).

`AuthContext.tsx`: This manages the user's identity. When a user logs in, their `studentSessionToken` is saved inside the browser's `sessionStorage`. The `AuthContext` wraps the entire application, meaning any component can instantly ask, "Is the user logged in? What is their name?" without having to talk to the backend again.

Design System: `index.css`

We use **Tailwind v4**. Instead of writing custom CSS for every button, Tailwind provides utility classes. The brand's signature color is a soft peach: `#F1BF98`. The typography uses *Inter* for standard text (highly readable) and *JetBrains Mono* for technical data like tokens and file sizes.

The Bridge: `config.ts`

This file contains a single, vital helper: `getApiUrl()`. When developers test the app on their laptop, the API lives at `localhost:3001`. But in production, it lives at a real web address. `config.ts` detects the environment and returns the correct URL, preventing the app from accidentally sending production data to a developer's local machine.

Chapter 15: Backend Architecture Deep Dive

First Principles: What is a Backend Server?

WHAT: The backend is a continuously running program that listens for network requests, enforces rules, talks to the database, and sends responses.

WHY: You can never trust the frontend. A malicious user could alter the frontend code to say "Print 1,000 pages for \$0.00." The backend is the unhackable authority that recalculates the math and verifies the permissions.

The Behemoth: `server/index.ts` (4,291 lines)

This single file is the brain of Campus Print. It is built on the *Express.js* framework. A web request traveling through this file goes through a "Middleware Stack"—a series of checkpoints.

The Middleware Pipeline:

1. `dotenv` : Loads secret passwords and API keys.
2. `Helmet (CSP)` : Adds security headers to prevent cross-site scripting attacks.
3. `CORS` : Ensures only requests from the official Campus Print website are allowed.
4. `compression` : Zips the data to make it travel faster over the internet.
5. `request logger` : Writes down every request ("User X asked for Page Y") for debugging.
6. `express.json()` : Translates the incoming network text into usable software objects.
7. `rate limiters` : Prevents spam. 120 requests/min for general API, 10/min for file uploads, 5/min for logins.

Authentication & Sessions

Not all users are equal. `index.ts` enforces a strict hierarchy:

- **Student Token:** Can only see and upload their own print jobs. Handled by the `requireAuth` middleware.
- **Shop Admin Token:** Can see the queue for their specific shop and approve jobs. Handled by `requireAdmin`.
- **Agent Token:** Used by the local `client.cjs` daemon. Allowed to download files and report printer status.

- **Owner Token:** God-mode. Can view revenue and manage shop settings.

Active sessions are stored in-memory using JavaScript `Map` objects (e.g., `activeOwnerSessions`). If a session is idle for too long, a timeout function automatically deletes it, forcing the user to log in again for security.

The SSE Router

The backend maintains an array called `sseClients`. Every time a browser or a print agent connects to the live stream, they are added to this list. Because firewalls often kill "quiet" connections, the server sends a tiny `keep-alive` comment down every tube every 15 seconds to keep the connection open.

Helper Functions

Scattered throughout the file are critical helpers like the `ConversionQueue`. If a student uploads a Word Document (.docx), it must be converted to a PDF before printing. Converting documents is heavy on the CPU, so the `ConversionQueue` limits the system to a maximum of 2 concurrent LibreOffice conversions to prevent the server from crashing.

Chapter 16: Database Design

First Principles: What is a Database?

WHAT: A database is a structured system for storing data so that it survives when the computer is turned off (persistence) and can be searched efficiently.

WHY: If we stored print jobs only in the server's RAM (memory), every time the server restarted for an update, all active print jobs, user accounts, and shop settings would be instantly vaporized.

The JSON Database: `db.json`

Most massive enterprise apps use complex databases like PostgreSQL. Campus Print currently uses a flat JSON file (`server/data/db.json`). Why?

Pros of JSON Database	Cons of JSON Database
No Setup: Works instantly on any machine without installing database server software.	Memory Limit: The entire database is loaded into RAM. If it grows to 5GB, the server crashes.
Human Readable: Developers can open the file in a text editor to easily debug issues.	No Indexing: Searching for a user means scanning every single record one by one.
Portable: Easy to backup by simply copying a single 3KB file.	No Concurrency: Only one write can happen at a exact millisecond. No transactions.

Atomic Writes: Preventing Corruption

If the server loses power exactly while writing to `db.json`, the file gets corrupted, destroying the entire system. To prevent this, the `writeDb()` function uses an *atomic write*. It writes the data to a temporary file (`db.json.tmp`). Once the write is 100% complete, it commands the operating system to instantly swap the temporary file for the real one. This guarantees the file is never left half-written.

The 7 Collections

1. **jobs (DbJob)**: The core record. Tracks `id`, `status` (e.g., queued, printing, completed, paper_empty), file size, cost, and the specific file paths.
2. **orders (DbPrintOrder)**: A grouping of jobs. If a student uploads 3 PDFs at once, they form 1 Order with a single `CP-XXXX` token.
3. **shops (Shop)**: Configuration for the physical locations (e.g., tjohn_print). Stores pricing (`bwPrice`, `colorPrice`), operating hours, and active printer IDs. (*The system auto-seeds 3 default shops if empty*).
4. **students (Student)**: User accounts created via Google OAuth. Tracks `email`, `picture`, and `lastSeen` time.
5. **agents (Agent)**: Telemetry for the `client.cjs` daemons. Tracks `machineName`, daemon version, and connection errors.
6. **printers (Printer)**: Hardware records. Tracks the physical names (e.g., "HP LaserJet 400") and discovery times.
7. **printerSettings (PrinterSettings)**: Maintenance overrides (e.g., marking a printer as "Out of Order" expected back at 2:00 PM).

Note on Heartbeats: While agents send a heartbeat every 10 seconds, writing to the JSON file every 10 seconds per shop would shred the hard drive. Heartbeats are tracked in-memory and only written to disk periodically or on state changes.

Chapter 17: File System & Storage Lifecycle

First Principles: How are Files Stored?

WHAT: Beyond just text data in the database, the system must handle the actual physical bytes of the PDF documents students upload.

WHY: We need strict rules for where these files go, how they are named to prevent mix-ups, and how they are deleted so the hard drive doesn't fill up and crash the server.

The Cloud Journey

1. The Upload (`server/uploads/`)

When a file arrives, the Multer middleware intercepts it. To prevent collisions (e.g., two students uploading "Homework.pdf" at the same time and overwriting each other), Multer generates a unique

filename using a timestamp plus 8 random hex bytes: `1723456789000-a1b2c3d4.pdf` . File sizes are hard-capped at 50MB.

Magic Bytes Validation: You cannot trust file extensions. A hacker could rename a malicious `virus.exe` to `virus.pdf` . Before saving, the system reads the first few bytes (the "magic bytes") of the file header to cryptographically prove it is a legitimate PDF document.

2. The Test Environment (`server/uploads-test/`)

When automated tests run, they upload dummy files. These are routed to an isolated `uploads-test` directory so they don't pollute the real production data.

3. Document Conversion (`conv-{timestamp}/profile`)

If LibreOffice is invoked to convert a file, it requires a "user profile". If two conversions happen at once using the same profile, LibreOffice crashes. The backend dynamically creates isolated, temporary profile directories for every single conversion job.

The Local Agent Journey

4. Local Download (`print-client/temp/`)

When the shop admin approves the job, `client.cjs` downloads the file from the cloud and saves it to its local `temp/` directory. This is where SumatraPDF reads it from.

5. Mock Printing (`print-client/printed_output/`)

For testing without wasting real ink and paper, the agent can be run in "mock mode". Instead of sending the PDF to the Windows Spooler, it simply moves the file into the `printed_output/` folder to simulate a successful print.

The Cleanup

Storage space is finite. Once a print job reaches the `completed` or `failed` status, the backend triggers a cleanup protocol. It looks up the `serverFilePath` and `originalFilePath` in the database and permanently deletes the physical files from the `uploads/` directory. The metadata remains in `db.json` for receipt purposes, but the heavy PDF bytes are gone forever.

Part III: CORE WORKFLOWS

Chapter 18: Student Registration & Login

What is OAuth?

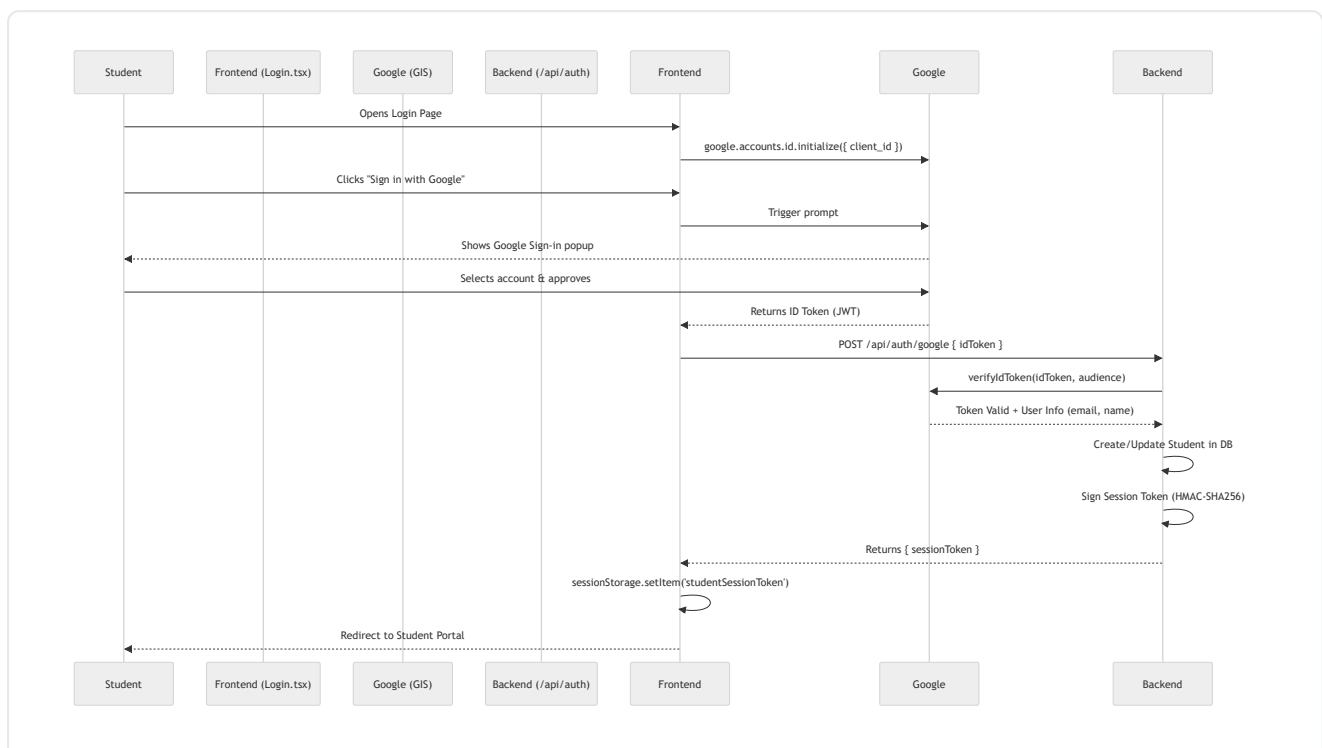
Authentication is the process of verifying who a user is. Historically, applications managed their own databases of usernames and passwords. This required developers to implement secure password hashing, salting, password reset emails, and account recovery workflows. It also required users to remember yet another password.

OAuth (Open Authorization) is a standard that allows users to grant third-party applications access to their information without sharing their passwords. In simpler terms: *OAuth means you let Google handle the password*. Instead of creating an account on Campus Print, a student uses their existing Google account to log in.

Documentation Conflict Note: You may notice the `index.html` title says "My Google AI Studio App" while the app is actually Campus Print. This is a known artifact from the original AI Studio boilerplate template that was never updated.

The Google OAuth Flow in Campus Print

The login process is implemented using the Google Identity Services (GIS) SDK. Here is the step-by-step flow from the moment a student opens the portal:



VERIFIED IMPLEMENTATION:

- `Login.tsx` initializes the SDK with `VITE_GOOGLE_CLIENT_ID` .
- The backend uses the official `google-auth-library` to verify the token signature and audience (preventing token reuse from other apps).

Session Tokens

Once Google verifies the user's identity, Campus Print issues its own session token. Why? Because the Google token expires quickly and we don't want to ping Google for every single API request the student makes.

The backend signs a JSON Web Token (JWT) using an HMAC-SHA256 signature with a secret key (`JWT_SECRET`). This token contains the student's ID and email. It is set to expire in exactly 24 hours (86,400,000ms).

Storage Strategy: The frontend stores this token in `sessionStorage` rather than `localStorage` .
Why? `sessionStorage` is automatically cleared when the browser tab or window is closed. This is crucial for shared campus computers where a student might forget to log out. If we used `localStorage` , the next student to use that computer would be logged into the previous student's account.

Mock Authentication

During development or testing environments where an internet connection or a Google Client ID might not be available, Campus Print supports a Mock Authentication mode.

When is it enabled? When the environment variable `ALLOW MOCK AUTH=true` is set, or in development mode if the token string starts with `mock_token_` .

Why does it exist? It allows developers to bypass the full Google OAuth flow and instantly simulate a logged-in student state to test the UI and document workflows.

Chapter 19: File Upload & Validation

The Drag-and-Drop Interface

The student portal utilizes HTML5 drag events (`dragenter` , `dragover` , `dragleave` , `drop`) to create a seamless file upload area. When a file is dropped, the browser exposes it via the `dataTransfer.files` object, passing it to our state management.

Frontend Validation

Before a file even touches the network, the frontend performs two critical checks:

1. **Extension Check:** Only `.pdf`, `.png`, `.jpg`, `.jpeg`, `.doc`, `.docx`, `.ppt`, `.pptx` are accepted.
2. **Size Check:** The file must not exceed 50MB.

If a file fails, an immediate error toast is shown, saving server bandwidth.

Multi-Layer Backend Security

Frontend validation is for UX; backend validation is for security. A malicious user could rename a virus from `malware.exe` to `malware.pdf`. To prevent this, the backend uses **Multer** (a Node.js middleware for handling `multipart/form-data`) and implements three layers of checks:

1. **Extension Check:** Matches the file extension.
2. **MIME Type Check:** Checks the `Content-Type` header sent by the browser.
3. **Magic Bytes Inspection:** The ultimate source of truth. The backend reads the first 8 bytes of the uploaded file to determine its actual format, regardless of extension.

File Type	Hex Signature (Magic Bytes)	ASCII Representation
PDF	25 50 44 46	%PDF
PNG	89 50 4E 47	%PNG
JPG / JPEG	FF D8 FF	ÿøÿ
DOCX / PPTX	50 4B 03 04	PK (ZIP format)
DOC / PPT	D0 CF 11 E0	ÐĬ (OLE2 format)

The ConversionQueue

When multiple files are uploaded, especially heavy Office documents, they are placed in a `ConversionQueue`. The queue is strictly limited to 2 concurrent LibreOffice processes. This prevents the server's CPU and RAM from being exhausted if 10 students upload 50MB PowerPoints at the exact same moment.

Chapter 20: Document Conversion Deep Dive

This chapter explores one of the most critical engineering challenges in Campus Print: accepting Microsoft Office files and reliably printing them.

The Evolution of Uploads

BEFORE: The system only accepted PDF files. It was simple and safe, but resulted in poor user experience. Students primarily work in Word or PowerPoint. Telling them to "convert it to PDF yourself before uploading" caused frustration and support requests.

THE PROBLEM: Students have `.docx` and `.pptx` files. The printing subsystem (SumatraPDF) natively prefers PDFs for reliable, headless rendering. We needed a server-side conversion mechanism.

Investigation of Conversion Tools

We evaluated several approaches:

- **Microsoft Office APIs:** Requires a licensed, graphical Office installation on the server. Windows-only. Too expensive and brittle for an automated server.
- **Google Docs API:** Cloud-dependent, requires uploading user data to external servers (privacy concerns), subject to API rate limits.
- **OnlyOffice:** Extremely complex to set up; the production version requires paid licenses.
- **Pandoc:** Great for text, but terrible for layout preservation. It would destroy complex PowerPoint slides.
- **LibreOffice Headless:** Free, open-source, runs on any OS, handles Office formats well, and supports a command-line `--convert-to pdf` mode.

DECISION: LibreOffice headless mode was selected.

The Windows Execution Problem (And Solution)

Implementing LibreOffice on Windows revealed a critical system-level bug.

When you spawn `soffice.exe` from Node.js, it acts as a launcher that spawns a background worker (`soffice.bin`) and then immediately exits. However, `soffice.bin` inherits the standard input/output (stdio) pipes. Because Node.js waits for the `close` event (which only fires when all stdio pipes are closed), the Node.js process would hang indefinitely waiting for the grandchild process.

THE FIX:

1. Use `soffice.com` (the console wrapper), NOT `soffice.exe` .
2. Listen to the `'exit'` event instead of the `'close'` event. The exit event fires as soon as the process terminates, ignoring lingering grandchild pipes.

Implementation: Isolated Profiles

If two LibreOffice processes run simultaneously using the same user profile, they will crash or corrupt each other. The backend creates an isolated, temporary profile directory for every single conversion:

```
const convOutputDir = `conv-${timestamp}-${randomId}`;
const profileDir = path.join(convOutputDir, 'profile');
// Spawns with: -env:UserInstallation=file:///${profileDir}
```

Conversions typically complete in 15-20 seconds on standard server hardware.

The LibreOffice Variance Problem

LibreOffice's text rendering engine reflows text slightly differently than Microsoft Word. A document that is exactly 40 pages in Word might spill over into 41 pages when converted to PDF by LibreOffice.

Billing Authority: The 41-page count from LibreOffice IS the correct billing count. Why? Because the generated PDF is exactly what gets sent to the physical printer. If 41 physical sheets of paper come out, the student must be billed for 41 pages.

Pre-conversion UX

While the server converts the file, the frontend shows a "Converting to PDF..." spinner. Once complete, the exact page count is displayed, and the `PdfFirstPageCanvas` component renders a preview using a local Vite-bundled PDF.js worker (avoiding external CDN dependencies).

Chapter 21: Page Counting

Why Page Count Matters

Accurate page counting is the financial bedrock of the system. If a student uploads a 100-page document but the system only bills for 1 page, the shop takes a massive loss on paper and ink.

Zero Trust Frontend: The frontend calculates the page count to show the user, but the backend *never* trusts the frontend's count. A malicious student could alter the frontend request to claim their 500-page book is only 1 page long. The backend always recalculates the final count.

Counting PDF Pages

The backend uses the `pdf-lib` library to parse the converted PDF:

```
const pdfDoc = await PDFDocument.load(buffer, { ignoreEncryption: true });
const pageCount = pdfDoc.getPageCount();
```

If `pdf-lib` fails (e.g., due to a malformed header), the system falls back to a regex search on the raw binary data looking for the PDF page count metadata tag: `/\/Count\s+(\d+)/g`.

Handling Page Ranges

Students don't always print the whole document. They can specify ranges like `"1-3,5,7-9"`.

The `countPagesFromRange(rangeStr, totalPages)` function parses this string, validates bounds against the total pages, expands the ranges, and places them into a JavaScript `Set` to automatically eliminate duplicates (e.g., `"1-5,3"` correctly counts as 5 pages, not 6).

Chapter 22: Billing & Pricing

The billing formula determines the `chargedAmount` for a job at the exact moment of submission. This value is locked into the `DbJob` record.

The Billing Formula

```
function calculateJobPrice(job, shop) {
  const rangeStr = job.pageRange || '';
  const printedPages = countPagesFromRange(rangeStr, job.pageCount);

  if (job.sides === 'double') {
    return job.copies * Math.ceil(printedPages / 2) * (shop.duplexPrice || 3);
  } else {
    const isColor = job.printType === 'color' || job.printMode === 'color';
    const rate = isColor ? (shop.colorPrice || 5) : (shop.bwPrice || 2);
    return job.copies * printedPages * rate;
  }
}
```

Worked Examples

Assume default shop prices: B&W = ₹2, Color = ₹5, Duplex = ₹3.

- **Example 1:** 10 pages, 2 copies, B&W, single-sided.
 $2 \text{ (copies)} \times 10 \text{ (pages)} \times ₹2 \text{ (rate)} = ₹40$
- **Example 2:** 10 pages, 1 copy, Color, single-sided.
 $1 \text{ (copy)} \times 10 \text{ (pages)} \times ₹5 \text{ (rate)} = ₹50$
- **Example 3:** 10 pages, 1 copy, B&W, double-sided.
 $1 \text{ (copy)} \times \text{ceil}(10/2) \text{ (sheets)} \times ₹3 \text{ (rate)} = 1 \times 5 \times ₹3 = ₹15$
- **Example 4:** 10-page doc, custom range "1-3,5", 1 copy, B&W, single-sided.
 Pages printed: 4. Formula: $1 \times 4 \times ₹2 = ₹8$

Shop owners can configure these baseline prices in their settings, allowing dynamic pricing based on their specific hardware costs.

Chapter 23: Admin Approval Workflow

The Token System

To keep references clean and user-friendly, Campus Print uses a dual-token system:

- **Order Token (`CP-XXXX`):** Represents the overall submission (e.g., a student uploading 3 files at once).

- **Job Token (`PRNT-XXXX`)**: Represents an individual file within that order.

Why Approval Exists

The queue defaults to `pending_approval`. This prevents abuse (students printing black blocks to waste toner) and allows the shop operator to control physical throughput. They can prioritize exams over personal notes, or pause approvals if paper is running low.

The Admin Interface

Admins log in using a `bcrypt` hashed password. (Legacy SHA-256 hashes are auto-upgraded upon successful login).

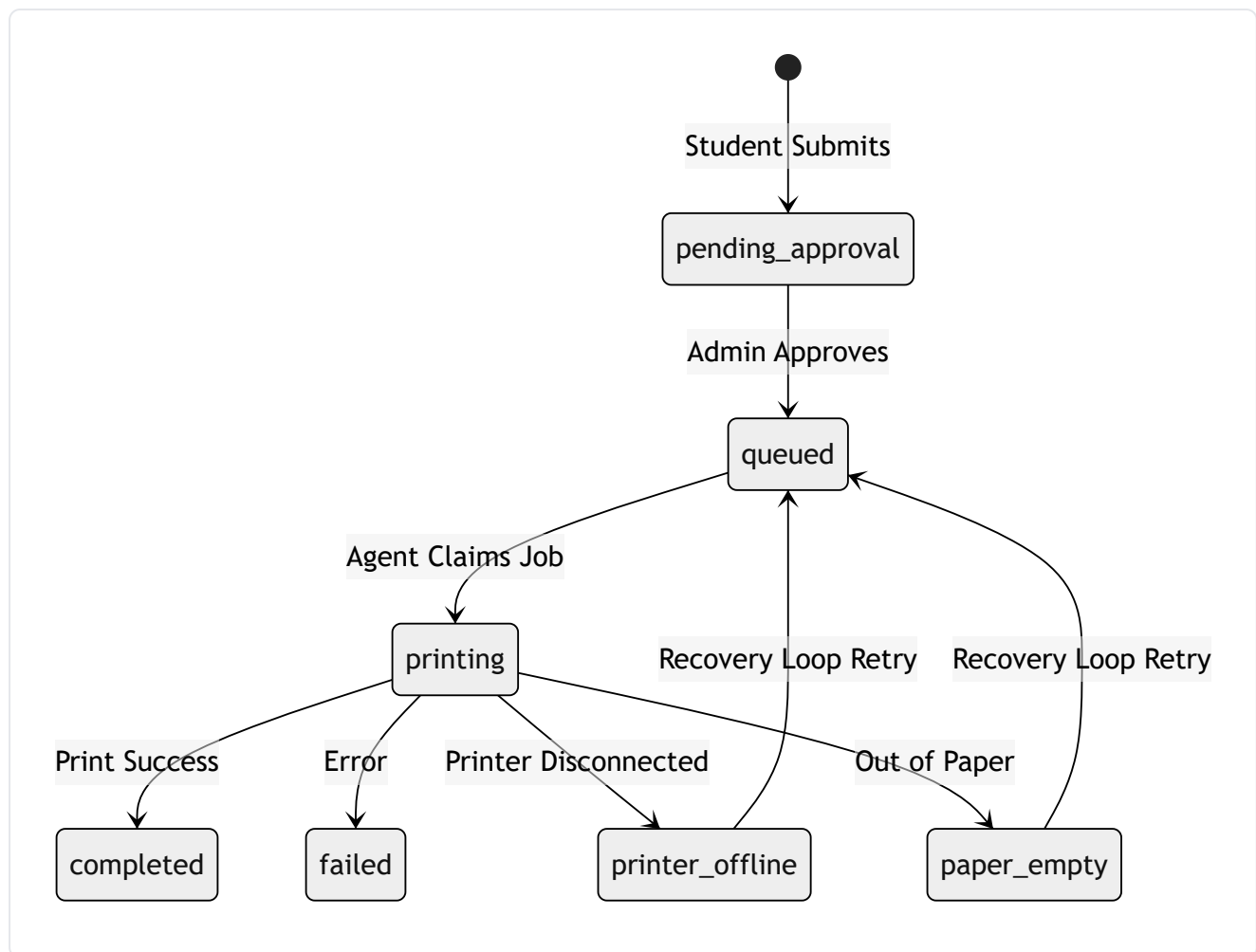
To ensure the server knows the admin portal is actively open, the frontend sends a 10-second heartbeat ping (`POST /api/auth/admin-ping`). The interface features a live search bar that queries `GET /api/orders/token/:query`, auto-completing after 3 characters.

When an admin clicks "Approve", the backend transitions all associated jobs to `queued` status and instantly triggers `dispatchNextJob()`.

Chapter 24: Queue System

A queue is a First-In-First-Out (FIFO) data structure. Campus Print requires a strict queue because multiple students can submit jobs simultaneously, but a physical printer can only process one file at a time.

Job State Machine



The Dispatch Logic

The core engine is the `dispatchNextJob(shopId)` function. When called, it evaluates:

1. **Agent Presence:** Is there an active SSE v2 connection for this `shopId` ?
2. **Concurrency:** Is there already a job in the `printing` state for this shop? (If yes, abort. Wait for completion).
3. **Printer Health:** Does the `agentHealthCache` show blocking states? (e.g., `OFFLINE` , `PAPER_JAM`).
4. **Selection:** Find the oldest job in the `queued` state whose `scheduledFor` timestamp is in the past.

If all conditions pass, the job is atomically marked as `printing` and pushed directly to the agent over the SSE stream.

Chapter 25: Server-Sent Events (SSE)

The Real-Time Communication Problem

When a job changes state, or a new job arrives, how do the Admin Portal and the Desktop Agent know without requiring the user to hit refresh?

- **Polling:** Client repeatedly asks "Any updates?". Wastes bandwidth and adds latency.
- **WebSockets:** Bidirectional, real-time, but requires complex proxy configurations (Nginx/Apache sometimes struggle) and a stateful protocol.
- **Server-Sent Events (SSE):** Unidirectional (server to client) over standard HTTP/1.1. Browsers have native auto-reconnect.

Decision: SSE was chosen because Campus Print only needs the server to push updates downstream. The clients use standard HTTP POST requests to send data upstream.

Implementation Details

The endpoint `GET /api/jobs/stream` establishes the connection with headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache
X-Accel-Buffering: no (Required for Nginx to not buffer the stream)
```

To prevent intermediate routers from dropping idle connections, the server sends a keep-alive comment every 15 seconds: `: keep-alive\n\n`.

The Milestone 8 Discovery: Early versions assumed that if an agent was marked "online" in the database, it could receive jobs. This caused lost jobs if the desktop PC lost Wi-Fi silently. The fix: jobs are ONLY dispatched if the server holds a live, active TCP socket for that specific agent in the `sseClients` array.

Broadcasting and Security

When `broadcastSse()` runs, it strips Personally Identifiable Information (PII) like `studentName`, `studentEmail`, and the `adminPasswordHash` before sending the JSON payload over the wire.

Chapter 26: Desktop Agent Deep Dive

The backend lives in the cloud. Physical printers live on Windows networks. The cloud cannot bypass corporate firewalls to speak to a local USB printer. Therefore, a Desktop Agent is required.

The 10-Stage Startup Sequence

Implemented in `client.cjs`:

1. **Single Instance Lock:** Creates `daemon.lock` to prevent multiple agents fighting for the same printer.
2. **Config Load:** Reads `config.json` (static) or `runtime.json` (dynamic protocol injection).
3. **Workspace Prep:** Creates `temp/` and `printed_output/` folders.
4. **Dependency Check:** Ensures `SumatraPDF.exe` exists (downloads from GitHub if missing).
5. **Printer Resolution:** Uses PowerShell to find the default Windows printer.
6. **Hardware Monitoring:** Initializes SNMP provider for toner/paper checks.
7. **Registration:** POSTs to `/api/agent/register`.
8. **Heartbeat:** Starts a 10s timer sending state payloads to the server.
9. **SSE Connection:** Connects to `/api/jobs/stream` as an agent.
10. **Graceful Shutdown:** Traps SIGINT/SIGTERM to cleanly disconnect.

The campusprint:// Protocol

To make installation foolproof, the Admin Portal has a "GO ONLINE" button. This triggers a custom Windows URL protocol: `campusprint://start?serverUrl=...&shopId=...&token=...`.

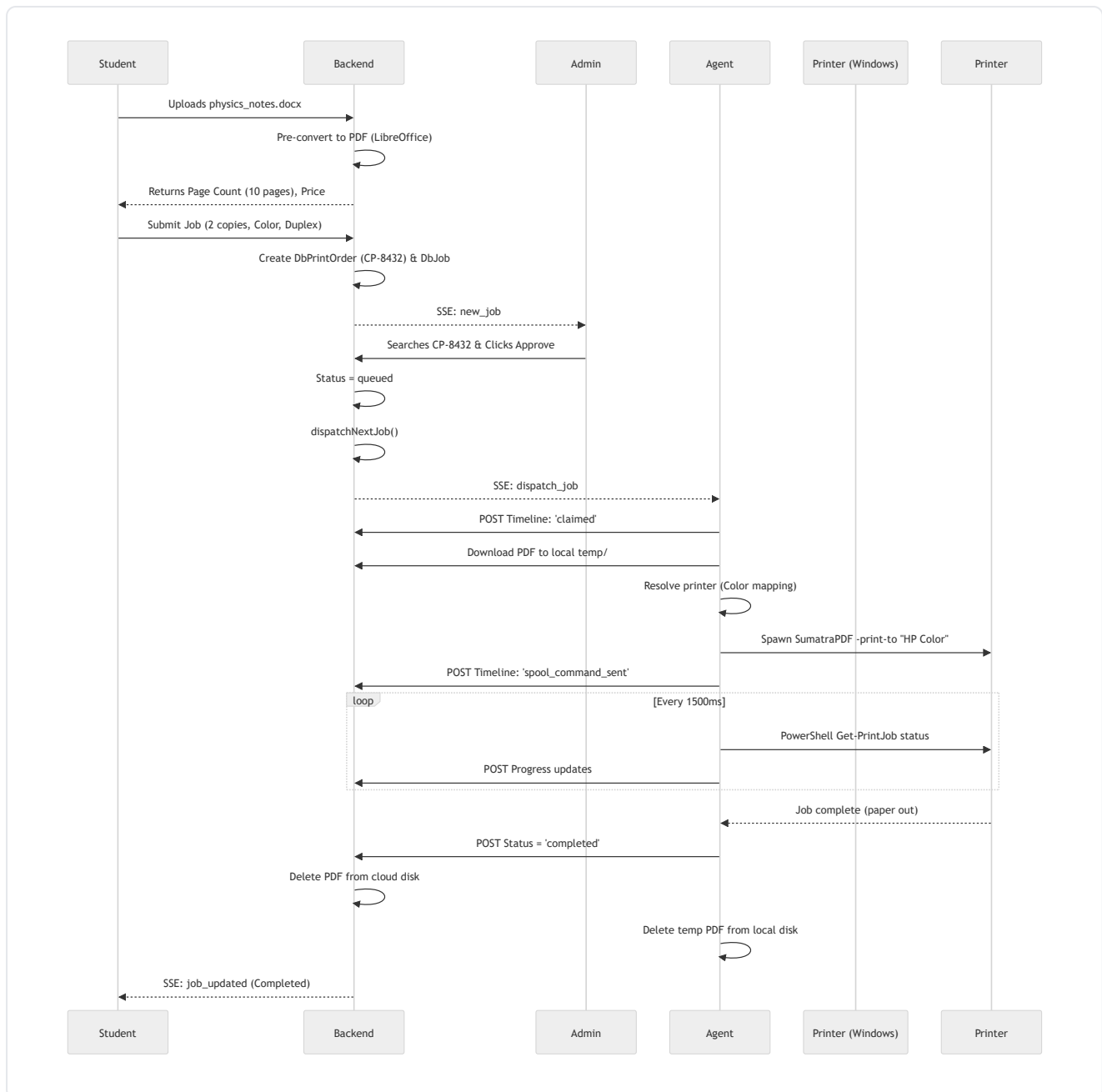
Windows intercepts this URL, launches `bridge.cjs`, which generates a `runtime.json` configuration file, and then seamlessly boots `client.cjs` in the background.

Mock Mode

If physical hardware isn't available, setting `mockPrinter=true` intercepts spool commands and writes the PDF output directly to the `printed_output/` directory, faking a 1500ms spooling delay.

Chapter 27: Complete Printing Pipeline (End-to-End)

This section traces a single document's journey from a student's laptop to physical paper.



The Narrative Flow

- 1. Upload:** A student drags `physics_notes.docx` into the portal.
- 2. Pre-Conversion:** The backend validates magic bytes, spawns an isolated LibreOffice profile, and converts it to PDF in ~15 seconds.
- 3. Configuration:** The frontend reads the 10-page count. The student selects 2 copies, Color, and Double-Sided. The formula calculates the price.
- 4. Submission:** The student clicks submit. The server creates order `CP-8432` and job `PRNT-F7A2`, defaulting to `pending_approval`.
- 5. Approval:** The admin sees the new job pop up via SSE. They review it and click Approve. The job moves to `queued`.

6. **Dispatch:** The server's `dispatchNextJob()` fires. It sees the desktop agent is online and idle. It pushes the job down the SSE tunnel.
7. **Processing:** The Desktop Agent receives the payload. It downloads the converted PDF. Based on the `Color` print requirement, it resolves the specific hardware printer name.
8. **Spooling:** The agent spawns headless SumatraPDF, instructing it to print duplex and color.
9. **Monitoring:** The agent polls the Windows Spooler every 1.5 seconds, sending progress percentage back to the server, which relays it live to the student and admin portals.
10. **Completion:** The physical paper drops into the tray. The agent reports completion, triggering the deletion of the temporary PDF files from both the cloud server and the local machine to protect student privacy.

Part IV: Printer Intelligence

Chapter 28: PrinterManager

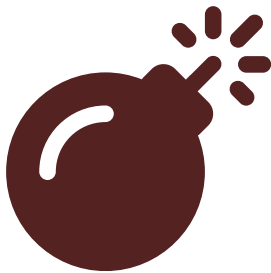
What Problem it Solves

In a distributed print shop environment, the backend server cannot see the physical printers. If the server dispatches a 50-page job to a printer that is out of paper, the job will hang, blocking the queue for all other students. To make smart dispatch decisions, the system needs real-time intelligence about the physical state of the hardware: paper levels, toner, jams, and offline status. The `PrinterManager` is the brain of the print client that provides this intelligence.

Architecture

The `PrinterManager` follows a modular architecture designed for extensibility. The core logic resides in `print-client/PrinterManager/index.cjs`, which orchestrates data collection. It depends on an abstract `IProvider` interface (`interfaces/IProvider.js`).

VERIFIED: Currently, the primary implementation of this interface is the `SnmpProvider` (`providers/SnmpProvider.cjs`), but the architecture allows for future providers (e.g., proprietary vendor APIs) without modifying the core manager.



Syntax error in text
mermaid version 10.9.8

The State Model

Regardless of the provider used, all hardware status is normalized into a unified `PrinterState` object (`models/PrinterState.cjs`).

```

{
  printerName: string,
  vendor: string | null,
  model: string | null,
  serialNumber: string | null,
  status: 'ready' | 'printing' | 'jam' | 'paper_empty' | 'offline' | 'cover_open' | 'error' |
  isOffline: boolean,
  isJam: boolean,
  isPaperEmpty: boolean,
  isCoverOpen: boolean,
  isPrinting: boolean,
  pageCount: number | null, // lifetime page counter
  consumables: [{ name, levelPct, unit }], // Crucial for toner tracking
  supportedFeatures: string[],
  provider: string,
  reachable: boolean,
  errorMessage: string | null,
  timestamp: string // ISO-8601
}

```

The `consumables` array is particularly important for shop operators, as it provides a standardized view of toner levels across different printer brands.

Adaptive Polling Strategy

A naive approach to monitoring would be to ask the printer for its status every second. However, this creates unnecessary UDP traffic and puts load on the printer's embedded processor. Conversely, polling every minute is too slow to promptly detect a paper jam or notify the backend when a print completes.

Campus Print solves this using an **adaptive polling strategy**. The polling interval dynamically adjusts based on the current state of the printer:

- **INTERVAL_FAST (3000ms):** Used when `status === 'printing'`. We need fast updates to detect job completion or immediate errors.
- **INTERVAL_MODERATE (15000ms):** The default state when idle or ready. The printer is stable, so less frequent checks are fine.
- **INTERVAL_SLOW (30000ms):** Used during known error states (jam, paper_empty, cover_open). The printer cannot accept jobs anyway, and human intervention is required, so we conserve resources.

Exponential Backoff

If the printer becomes unreachable (e.g., network drop, power off), the manager implements an exponential backoff to prevent flooding the network with failing requests.

VERIFIED Formula: `delay = min(60000, 10000 × 2(retryCount - 1))`

The sequence is: 10s, 20s, 40s, capped at a maximum of 60s (`INTERVAL_BACKOFF_MAX`). Health monitoring data is exposed via `getMetrics()` , which returns `{ retryCount, consecutiveFailures, lastSuccessfulPoll, isPolling }` .

This intelligence directly integrates with `dispatchNextJob` in the client. Blocked states (like jam or paper_empty) prevent the agent from claiming new jobs for that specific printer, routing them to healthy printers instead.

Chapter 29: SNMP from Zero

What is a Network Protocol?

At its core, a network protocol is just an agreed-upon set of rules for how two computers exchange data. It defines the format of the messages and the sequence of interactions. To understand how Campus Print talks to printers, we must start with the transport layer.

UDP vs TCP

Most web traffic uses TCP (Transmission Control Protocol). TCP requires a "handshake" to establish a connection and guarantees delivery by acknowledging every packet. This makes it reliable but slow.

UDP (User Datagram Protocol), on the other hand, is "fire-and-forget". It sends the data without a handshake and doesn't wait for acknowledgment. It's extremely fast and lightweight.

SNMP operates over UDP on **Port 161**. Just as a web server listens on port 80 or 443, a printer's management agent listens on UDP port 161.

What is SNMP?

Simple Network Management Protocol (SNMP) is a standard way to query network-attached devices for their status. It is supported by nearly every enterprise and prosumer printer on the market.

Information in SNMP is organized in a **MIB (Management Information Base)**. Think of a MIB as a virtual filesystem containing data points instead of files. Each data point is addressed by an **OID (Object Identifier)**. An OID looks like an IP address but much longer, representing a path in the MIB tree.

Example: `1.3.6.1.2.1.1.1.0` represents the "System Description".

SNMP v2c and BER/DER Encoding

Campus Print uses SNMP version 2c. It is widely supported and uses a simple "community string" as a password (by default, usually 'public' for read-only access). To send a request, we construct a packet containing the OID we want and the community string. This packet is formatted using **BER (Basic Encoding Rules)** or its subset DER, which defines exactly how to translate integers, strings, and OIDs into raw binary bytes.

The Ponytail Philosophy: Zero Dependencies

Why did Campus Print write a custom SNMP implementation instead of using an `npm` package? Most SNMP libraries in the Node ecosystem rely on native C/C++ addons. These addons must be compiled for the specific operating system and Node version running on the machine. This introduces massive friction for deployment—shop computers would need Python and build tools installed just to run the print client.

The "Ponytail Philosophy" favors the standard library over external dependencies. Since Campus Print only needs to perform simple GET requests (no SET or bulk operations), the engineering team built a pure-JavaScript implementation using Node's built-in `dgram` (UDP) module. It is robust, lightweight, and runs anywhere Node.js runs.

Chapter 30: SnmpProvider Deep Dive

The 9 Critical OIDs

The `SnmpProvider.cjs` implementation specifically queries 9 OIDs to build the `PrinterState` :

1. `1.3.6.1.2.1.1.1.0` : System Description (vendor/model string)
2. `1.3.6.1.2.1.1.3.0` : System Uptime
3. `1.3.6.1.2.1.25.3.5.1.1.1` : HR Printer Status (3=ready, 4=printing, 5=other/error)
4. `1.3.6.1.2.1.25.3.5.1.2.1` : HR Printer Detected Error State (bitmap)
5. `1.3.6.1.2.1.43.5.1.1.17.1` : Printer Serial Number
6. `1.3.6.1.2.1.43.10.2.1.4.1.1` : Lifetime Page Counter
7. `1.3.6.1.2.1.43.11.1.1.9.1.1` : Toner Current Level
8. `1.3.6.1.2.1.43.11.1.1.8.1.1` : Toner Max Capacity
9. `1.3.6.1.2.1.43.12.1.1.4.1.1` : Consumable Description

Request and Response Decoding

The provider BER-encodes the OID and wraps it in an SNMP v2c GET-REQUEST packet. When the printer responds over UDP, the data arrives as a binary Buffer. The code parses this Buffer using TLV (Type-Length-Value) logic. Every piece of data is prefixed with its type (e.g., Integer, String) and its length in bytes, allowing the parser to walk through the binary payload safely.

Parsing the Error Bitmap

The most complex OID is the "HR Printer Detected Error State". It returns a single byte that acts as a bitmap, where each bit represents a specific error condition.


```

Bit Position (Byte 0):
[7] [6] [5] [4] [3] [2] [1] [0]
|   |   |   |   |   |   |
|   |   |   |   |   |   +-- isLowPaper
|   |   |   |   |   +----- isOffline (0x02)
|   |   |   |   +----- isJam (0x04)
|   |   |   +----- isCoverOpen (0x08)
|   |   +----- isOutputBinFull
|   +----- isLowToner (0x20)
| +----- isPaperEmpty (0x40)
+----- isDoorOpen

```

For example, if the byte is `01000110` (Hex 0x46), it means bits 6, 2, and 1 are set. The printer is Paper Empty, Jammed, and Offline.

Printer IP Discovery

To communicate via SNMP, the provider needs the printer's IP address. Printers installed on Windows are referenced by name, not IP. To bridge this gap, Campus Print executes a PowerShell command:

`Get-PrinterPort` to find the `PrinterHostAddress` associated with the printer's logical port. If the printer is local (USB), SNMP is skipped.

Fallback Behavior: If SNMP fails, times out (after 4000ms), or the printer lacks network capability, the provider calls `createUnreachableState()`. The client then falls back to relying entirely on the local Windows Spooler status.

Scope of Implementation

IMPLEMENTED: Core status, paper empty, jam, cover open, offline, toner level (calculated from current/max), serial number, page count, and system description.

NOT IMPLEMENTED: Specific paper tray levels (too variable across vendors), detailed proprietary error codes, and vendor-specific OIDs.

Chapter 31: Windows Printing Stack

The Windows Print Spooler

When an application prints on Windows, it doesn't talk directly to the hardware. Instead, it generates a graphical representation of the document and submits it to the **Windows Print Spooler**. The Spooler manages a queue of jobs, queuing them on disk, and feeding them to the printer driver at the speed the printer can handle.

WMI and Win32_Printer

Campus Print interacts with the spooler using PowerShell and WMI (Windows Management Instrumentation). The `Win32_Printer` class provides high-level status. Properties like `PrinterStatus` (3=Idle, 4=Printing), `WorkOffline`, and `DetectedErrorState` are queried to augment the SNMP data, especially for USB printers.

SumatraPDF Integration

To actually print a PDF without user interaction, the client uses **SumatraPDF**. It is a lightweight, open-source PDF viewer that supports command-line printing. The agent spawns a child process:

```
cp.spawn(sumatraPath, ['-print-to', resolvedPrinterName, '-print-settings', printSettings, file])
```

The `printSettings` string dictates the format: `fit,<color|monochrome>,<duplexlong|simplex>,<N>x`. Using `windowsHide: true` is critical to prevent console windows from flashing on the shop operator's screen during headless operations.

Monitoring the Spooler Queue

Once SumatraPDF submits the job to the spooler, the client must monitor its progress. It enters a polling loop (every 1500ms), executing `Get-PrintJob` via PowerShell. It checks the `PagesPrinted` versus `TotalPages` properties.

Stuck Job Detection: If `PagesPrinted` remains unchanged for 40 consecutive checks (60 seconds), the client assumes the job is permanently stuck in the spooler (perhaps due to a silent driver error) and flags it as an error to alert the operator.

Part V: Security & Authentication

Chapter 32: Authentication System

Authentication vs Authorization

Authentication (AuthN) is the process of proving *who you are*. **Authorization (AuthZ)** is proving you are *allowed to do something*. Campus Print rigorously separates these concepts to enforce security boundaries.

Passwords and bcrypt

We never store passwords in plaintext. Instead, we store a hash. However, fast hashing algorithms (like MD5 or SHA-256) are vulnerable to brute-force attacks. Campus Print uses **bcrypt**, which is

intentionally slow and automatically includes a unique "salt" for every password, rendering rainbow tables useless.

Tokens and Stateless Sessions

Instead of storing active sessions in a database, Campus Print uses stateless tokens. While similar to JSON Web Tokens (JWT), the system uses a custom HMAC-SHA256 token format. The payload is Base64Url encoded, and the server appends a cryptographic signature. When the client sends the token back, the server recalculates the signature using its secret key. If they match, the token is valid. If tampered with, the signature invalidates.

The 5 Authentication Mechanisms

1. **Google OAuth (Students):** Students authenticate via Google. The frontend sends an `idToken` to `/api/auth/google`. The backend verifies this JWT with Google, creates a student record, and issues a 24-hour HMAC session token.
2. **Student Session Tokens:** Standard web sessions using the HMAC-SHA256 tokens stored in the browser's `sessionStorage`. Middleware (`requireAuth`) verifies them on every request.
3. **Admin bcrypt Login:** Shop admins log in via `/api/auth/login` providing a shopId, username, and password. The system checks `bcrypt.compare`. It prevents duplicate simultaneous logins with a 30-second window and returns a signed shop token (30-min expiry).
4. **Agent Tokens:** The headless print client authenticates using a static `AGENT_TOKEN` header or a long-lived (10-year) signed shop token.
5. **Mock Auth:** Used strictly for development. Bypasses Google OAuth by accepting tokens starting with `mock_token_`. **Must never be enabled in production.**

Security Boundaries

- **Students:** Can only submit jobs and view their own orders.
- **Shop Admins:** Can only view and manage jobs belonging to their specific `shopId`. This prevents cross-tenant data leaks.
- **Platform Owner:** A special role that can view telemetry across all shops but in a read-only capacity.
- **Agent:** Authorized only to claim jobs and update statuses for its assigned `shopId`.

Chapter 33: Security Hardening

Helmet and HTTP Headers

The Express backend uses Helmet to set secure HTTP headers. It removes the `X-Powered-By` header to obscure the technology stack. More importantly, it implements a strict **Content Security Policy (CSP)**.

A CSP tells the browser exactly which external resources are allowed to load. For example, `imgSrc: ['self', data:, blob:, lh3.googleusercontent.com]` ensures that images can only be loaded from the

local server, inline data, or Google (for student avatars). The `scriptSrc` includes `unsafe-inline` to accommodate Tailwind CSS, which injects inline styles.

CORS (Cross-Origin Resource Sharing)

Browsers enforce a same-origin policy, preventing a script on `site-a.com` from fetching data from `site-b.com`. CORS is a mechanism to selectively bypass this. Campus Print uses a dynamic origin check function that allows requests from localhost (for development and local agent interaction) and domains explicitly listed in the `ALLOWED_ORIGINS` environment variable.

Rate Limiting

To protect against Denial of Service (DoS) and brute-force attacks, the system employs aggressive rate limiting:

- **API Limiter:** 120 requests per minute per IP.
- **Upload Limiter:** 10 requests per minute (prevents disk exhaustion).
- **Login Limiter:** 5 requests per minute (slows down credential stuffing).

Note: `localhost` is exempted from these limits to allow automated test suites to run at full speed without being blocked.

Chapter 34: File Security

Validating Uploads

When a user uploads a file, trusting the file extension (e.g., `.pdf`) is the weakest form of security. An attacker can easily rename `malware.exe` to `document.pdf`. Checking the MIME type provided by the browser is slightly better, but still client-controlled.

The strongest check is examining the **Magic Bytes**—the first few bytes of the file content itself. A valid PDF file will always begin with the hex values `25 50 44 46` (which translates to ASCII `%PDF`). This cannot be faked without corrupting the file's structure.

Command Injection Prevention

Because the agent executes PowerShell commands containing printer names, it is vulnerable to command injection. If a malicious user named a printer `printer; rm -rf /`, a naive implementation would execute the deletion command. Campus Print prevents this using `sanitizeCmdArg()`, which strips dangerous characters using the regex `/[;&|`$"'\\>\n\r]/g`.

Path Traversal

If an API endpoint accepts a filename like `../../../../etc/passwd`, it could expose sensitive server files. All user-supplied filenames are passed through Node's `path.basename()`, which strips all directory

components, reducing the input to just the file itself.

Broadcast Sanitization

VERIFIED: Real-time Server-Sent Events (SSE) broadcast job updates to the shop dashboard. To prevent leaking Personally Identifiable Information (PII), the `broadcastSse()` function explicitly strips `studentName`, `studentEmail`, `serverFilePath`, `tokenId`, and password hashes before transmitting the payload to the frontend.

Chapter 35: Security Evolution

Security is not a state, but a process. Campus Print has evolved significantly from its early prototypes.

HISTORICAL TIMELINE:

- **Pre-RC1: Static Admin Tokens.** The system initially used a shared static token for all admin access. This was a massive vulnerability; if the token leaked, all shops were compromised. This was replaced with HMAC-signed per-shop tokens with a strict expiration.
- **Password Storage Upgrade.** Passwords were originally hashed with unsalted SHA-256. Because SHA-256 is designed to be fast, it is highly susceptible to brute-forcing. The system was upgraded to bcrypt. A seamless migration path allows legacy SHA-256 users to log in, whereupon their password is automatically re-hashed with bcrypt.
- **Command Injection Mitigations.** As PowerShell integration deepened, the risk of shell injection became apparent. The strict `sanitizeCmdArg()` function and printer name regex validation (`/^[a-zA-Z0-9 _.-]+$/`) were implemented to neutralize this threat vector.
- **Mock Auth Guards.** Early iterations allowed `ALLOW MOCK AUTH=true` to bypass Google OAuth unconditionally. Strict guards were added so that mock auth is physically rejected in production environments, preventing accidental exposure.
- **Billing Manipulation.** Initial prototypes calculated job prices on the frontend. A malicious user could theoretically alter the network request to send a lower price. This was corrected by making the backend authoritative: the server calculates the price using `calculateJobPrice()` and ignores any cost data submitted by the client.
- **SSE Data Minimization.** It was discovered that real-time job broadcasts included student PII and internal server paths. The broadcast function was patched to aggressively sanitize the payload before transmission.

Chapter 31B: If I Had to Build Campus Print Again (27-Step Engineering Blueprint)

 **Practical Blueprint:** This chapter presents the complete, step-by-step engineering roadmap for building Campus Print from scratch. If you had to reconstruct this system from zero tomorrow, follow these 27 sequential steps.

Phase 1: Foundation & Specs

1. **Problem Definition:** Identify real pain points — USB drive virus risks at print shops, manual counter queues, lack of exact pricing visibility, paper waste.
2. **Requirements & Boundaries:** Define clear scope: Web app for students/admins, local daemon for shop PC, PDF conversion pipeline, atomic JSON database, zero cloud deployment costs for testing.
3. **Architecture Design:** Establish component boundaries — Frontend (Vite/React), Backend (Express/Node.js), Desktop Agent (Node.js CJS daemon), Database (flat-file JSON with atomic writes).
4. **Repository Setup:** Initialize Git, set up folder structure (`src/` , `server/` , `print-client/` , `tests/`), configure `.gitignore` , `package.json` , and `tsconfig.json` .

Phase 2: Core Web Application

5. **Frontend Scaffold:** Set up React with Vite and Tailwind CSS v4. Implement design system variables in `src/index.css` featuring brand color `#F1BF98` .
6. **Backend Server Scaffold:** Create Express server in `server/index.ts` . Add Helmet security headers, CORS origin filtering, rate limiters, and compression middleware.
7. **Database Layer:** Implement `server/db.ts` with LowDB-style flat JSON storage. Implement atomic write strategy (write to `.tmp` file then atomic rename) to prevent file corruption.
8. **Authentication System:** Integrate Google Identity Services (GIS) on frontend. Implement `/api/auth/google` verifying Google ID tokens via `google-auth-library` . Generate HMAC-SHA256 session tokens. Add bcrypt password hashing for shop admins.

Phase 3: File Processing & Billing

9. **Multipart Upload Pipeline:** Configure Multer in `server/index.ts` with 50MB size limit. Implement strict 3-tier validation (file extension + MIME type + 8-byte magic byte signature verification).
10. **LibreOffice Pre-Conversion Pipeline:** Implement `convertOfficeToPdf()` using LibreOffice headless CLI (`soffice.com` wrapper on Windows) with isolated user profile directories (`conv-*`) and a 2-process concurrency queue.
11. **Page Counting Engine:** Implement `countPdfPages()` using `pdf-lib` with binary regex fallback (`/\\/Count\\s+(\\d+)/g`). Implement custom page range parser `countPagesFromRange()` .
12. **Authoritative Billing Engine:** Implement `calculateJobPrice()` on backend. Compute total price based on printed pages, copies, print mode (B&W vs. Color), and side mode (single vs. double/duplex). Never trust client-sent prices.

Phase 4: Order Queue & Real-Time Sync

13. **Admin Approval Queue:** Build `AdminPortal.tsx` featuring live autocomplete order token search (`CP-XXXX`). Implement single and batch order approval endpoints (`POST /api/orders/:id/approve`).
14. **Queue State Machine:** Implement job status transitions: `pending_approval` → `queued` → `printing` → `completed` / `failed` .

15. **Server-Sent Events (SSE) Engine:** Build `GET /api/jobs/stream` endpoint with 15-second keep-alive comments. Implement `broadcastSse()` stripping student PII before broadcasting. Connect frontend `App.tsx` to SSE stream with 5-second polling fallback.

Phase 5: Desktop Print Agent & Spooler

16. **Windows Agent Scaffold:** Build `print-client/client.cjs` daemon with 10-stage startup sequence, single-instance lockfile enforcement (`daemon.lock`), and 10-second heartbeat loop (`POST /api/agent/heartbeat`).
17. **Protocol Launcher Handler:** Build `bridge.cjs` and `Register_Protocol.bat` registering custom OS URI handler `campusprint://` to allow Admin Portal to trigger agent connection.
18. **Silent SumatraPDF Spooling:** Integrate `SumatraPDF.exe` CLI with `windowsHide: true` process flags. Implement dynamic printer resolution (`resolvePrinterForJob()`) matching jobs to B&W or Color printers.
19. **Windows Print Spooler Monitoring:** Query WMI / PowerShell `Get-PrintJob` every 1500ms to monitor page progress (0-100%) and detect stuck jobs or offline hardware. Implemented recovery polling loops.

Phase 6: Hardware Intelligence & Operations

20. **PrinterManager & SNMP v2c:** Build `SnmpProvider.cjs` using pure Node.js UDP sockets (`dgram` on port 161) with custom BER/DER encoders/decoders. Query 9 standard printer OIDs and decode error bitmaps (paper empty, jam, cover open, offline, toner level).
21. **Adaptive Polling Scheduler:** Implement dynamic polling intervals: 3s while printing, 15s while idle, 30s during errors, with exponential backoff on connection failure.
22. **Security Hardening Audit:** Sanitize shell arguments (`sanitizeCmdArg()`), enforce shop scoping checks on all endpoints, and add `path.basename()` traversal guards.
23. **Automated Test Suites:** Write Vitest unit/API tests (18 test files) and Playwright E2E specs for happy path, validation, and conversion pipelines.
24. **Local Verification:** Run backend, frontend, agent daemon, and LibreOffice on local machine. Submit test DOCX, verify PDF conversion, page count, pricing, approval, SSE dispatch, and spooling.
25. **Resource Measurement:** Measure RAM/CPU usage during LibreOffice conversion (~200MB RAM, 100% CPU spike for 15s) and SSE bandwidth consumption.
26. **Cloudflare Tunneling:** Configure Cloudflare Tunnel (`cloudflared`) to expose local backend port 3001 to public HTTPS URL for Google OAuth callback testing. Deploy frontend static build to Cloudflare Pages (`wrangler.json`).
27. **Production Operations & Backup Strategy:** Set up automated `db.json` file backups, configure production environment variables, disable mock auth, and monitor system health.

Part VI: Engineering Operations

Chapter 36: Testing

What is software testing? In the context of software engineering, testing is the process of evaluating a system or its components to determine whether they satisfy the specified requirements. Testing helps us find bugs before our users do. Think of it as a series of automated checks that ensure your code behaves exactly as expected.

There are several types of testing we employ in Campus Print:

- **Unit Testing:** Testing a single function or component in isolation. For example, testing if a function correctly calculates the price of a print job.
- **Integration Testing:** Testing how multiple components work together. For instance, testing if the API correctly processes a document, interacts with the database, and returns the expected response.
- **End-to-End (E2E) Testing:** Testing the entire system from the user's perspective. This involves simulating a user clicking through the web interface, submitting a job, and verifying the final outcome.

Why do we test? Writing tests builds confidence in our changes. When you add a new feature or fix a bug, running the automated tests ensures you haven't accidentally broken existing functionality—a scenario known as a regression.

Vitest: Our Unit and Integration Testing Framework

Vitest is a blazing fast unit test framework powered by Vite. It is compatible with Jest (another popular testing framework) but is optimized for projects using Vite, like our frontend.

In Vitest, tests are structured using a few key functions:

- `describe()` : Groups related tests together.
- `it()` or `test()` : Defines a single test case.
- `expect()` : Makes an assertion about a value (e.g., `expect(result).toBe(true)`).

VERIFIED: Vitest Configuration

Our Vitest workspace configures three distinct environments:

- `frontend` : Uses `jsdom` to simulate a browser environment for React components (`tests/unit/frontend/**`).
- `backend` : Uses `node` for Express API tests (`tests/unit/backend/**` , `tests/api/**`).
- `agent` : Uses `node` for the Desktop Agent tests (`tests/unit/agent/**` , `tests/integration/agent/**`).

Crucially, we set `fileParallelism: false`. Why? Because our current persistence layer uses a local JSON file (`db.test.json`). If multiple tests run in parallel, they would attempt to read and write to this file simultaneously, causing file lock conflicts and flaky test failures.

Playwright: End-to-End Testing

Playwright is an automated browser testing framework. It physically launches a real browser (Chromium, Firefox, or WebKit), navigates to our application, and interacts with elements just like a human user would.

VERIFIED: Playwright Configuration

- **Test Directory:** `./tests/e2e`
- **Timeout:** 30,000ms
- **Workers:** 1 (Similarly to Vitest, running sequentially prevents concurrent database write locks).
- **Auto-starts:** The configuration automatically starts the backend server (`tsx server/index.ts`) on port 3001 and the Vite frontend on port 3000 using `NODE_ENV=test`.

Documenting the Test Suite

Here are the verified test files in our repository and what they cover:

Test File	Type	What It Tests
<code>tests/api/backend.test.ts</code>	API	Auth middleware, CORS, shop lifecycle, agent registration, payload compression, and rate limiting. Ensures core API stability.
<code>tests/api/agent/register-401.test.ts</code>	API	Verifies that an agent attempting to register without proper authorization receives a 401 Unauthorized response.
<code>tests/api/agent/register-success.test.ts</code>	API	Verifies the happy path for desktop agent registration and token issuance.
<code>tests/api/conversion.test.ts</code>	API	Tests PDF parsing, LibreOffice DOCX conversion, and image conversion. <i>VERIFIED RESULT: 5 conversion tests pass in 29.88s (DOCX conversion alone takes ~28s).</i>
<code>tests/api/jobs.test.ts</code>	API	Tests the full job lifecycle (creation, state transitions, completion) ensuring the state machine holds.
<code>tests/api/printers.test.ts</code>	API	Tests remote printer discovery, capabilities mapping, and routing logic based on shop isolation rules.
<code>tests/e2e/admin-agent-validation.spec.ts</code>	E2E	Simulates admin UI interactions to validate connected agents.
<code>tests/e2e/admin-approval-workflow.spec.ts</code>	E2E	Tests the CP-XXXX token-based approval workflow from the Admin dashboard.
<code>tests/e2e/admin-force-complete.spec.ts</code>	E2E	Verifies admins can override job states and force completion.
<code>tests/e2e/admin-reject-workflow.spec.ts</code>	E2E	Verifies the job rejection process and associated cleanup.
<code>tests/e2e/connection/go-online.spec.ts</code>	E2E	Tests the agent going online and syncing heartbeat status via SSE.
<code>tests/e2e/connection/launcher-handoff.spec.ts</code>	E2E	Verifies the handoff mechanism between the system launcher and the agent process.
<code>tests/e2e/connection/runtime-file.spec.ts</code>	E2E	Ensures the agent correctly locks the runtime file to prevent multiple instances.
<code>tests/e2e/pause-retry-workflow.spec.ts</code>	E2E	Simulates print failures and verifies the pause/retry mechanisms.
<code>tests/e2e/student-happy-path.spec.ts</code>	E2E	The golden path: student uploads, configures, and submits a job successfully.

Test File	Type	What It Tests
<code>tests/e2e/student-validation.spec.ts</code>	E2E	Tests frontend validation (e.g., clamping copies between 1-10).
<code>tests/e2e/token-search-workflow.spec.ts</code>	E2E	Tests searching for jobs via CP-XXXX tokens in the admin view.
<code>tests/unit/placeholder.test.ts</code>	Unit	Placeholder test for basic CI integration.

Running Tests

You can execute these tests via predefined NPM scripts:

```
npm test           # Runs unit + API tests sequentially
npm run test:api   # Runs API integration tests only
npm run test:unit  # Runs unit tests only
npm run test:e2e   # Runs Playwright E2E tests
npm run lint       # Runs TypeScript type checking (tsc --noEmit)
npm run build      # Verifies the production Vite build works
```

Chapter 37: Git & Version Control

What is version control? Imagine Microsoft Word's "Track Changes", but for thousands of files across an entire engineering team. Version control allows you to save snapshots of your code, track who made what changes, and easily revert mistakes. Git is the world's most popular version control system.

- **Repository:** The project folder that is actively tracked by Git.
- **Commit:** A permanent snapshot of your files at a specific point in time, accompanied by a descriptive message.
- **Branch:** An independent line of development. You can work on a new feature in a separate branch without affecting the stable "main" code.
- **Remote:** A version of your repository hosted on a server (like GitHub) that allows multiple developers to collaborate.

Safe Git Commands for Campus Print

```
git status          # See which files have changed, been added, or deleted
git diff            # See the exact line-by-line changes you've made
git add .           # Stage ALL changed files for the next commit
git add server/index.ts  # Stage only a specific file
git commit -m "Fix: auth middleware now verifies shopId" # Save the snapshot
git push            # Send your commits to the remote repository
git pull            # Fetch and merge the latest changes from the remote
git log --oneline -10 # View the last 10 commits compactly
git checkout -- src/App.tsx # DANGER: Discard uncommitted changes to a specific file
```

VERIFIED: The Campus Print .gitignore Explained

A `.gitignore` file tells Git which files it should NEVER track. This is critical for security and repository health. Our configuration ignores:

- `node_modules/` , `build/` , `dist/` : Generated files. They can be rebuilt anytime using `npm install` and `npm run build` .
- `server/data/` (including `db.json`): Contains live application data, not source code.
- `server/uploads/` , `server/uploads-test/` : Contains large, temporary student files that would bloat the repository.
- `print-client/temp/` , `print-client/printed_output/` : Local temp directories for agent file processing.
- `.env*` (except `.env.example`): **Never commit secrets.** These files contain API keys, database passwords, and JWT secrets.
- `print-client/config.json` : Contains agent authentication tokens.
- `*.exe` , `*.zip` : Large binary files that do not belong in source control.
- `test-results/` , `playwright-report/` : Output from end-to-end tests.
- `.agents/` , `.aider*` , `print-client/shutdown.signal` : Miscellaneous logs and signals.

What IS committed: Source code, tests, documentation, and configuration like `package.json` .

Chapter 38: Local Development Guide

This chapter provides a complete, step-by-step guide to setting up Campus Print on your local machine.

Prerequisites

- **Node.js (v18+)**: The runtime environment for both the backend and frontend build tools.
- **LibreOffice (v26.2.5.2+)**: Required by the backend for converting DOCX and other formats to PDF for printing. (Verified installation path: `D:\LibreOffice\`)
- **Git**: For version control.

- **Google Cloud Project:** With OAuth credentials configured for user authentication.

Initial Setup

1. Clone the repository to your local machine.
2. Run `npm install` in the root directory. This installs all dependencies (18 runtime, 15 dev).
3. Copy `.env.example` to `.env`.
4. Fill in the `.env` variables:
 - `GOOGLE_CLIENT_ID` : From your Google Cloud Console.
 - `JWT_SECRET` : A long random string (min 32 chars) for signing authentication tokens.
 - `ADMIN_API_KEY` : A secret key for administrative access.
 - `AGENT_TOKEN` : A secret token used for the desktop agent.
 - `OWNER_PASSWORD` : A strong password.
 - `LIBREOFFICE_PATH` : Absolute path to the executable (e.g., `D:\LibreOffice\program\soffice.exe`).

Starting the Servers

You need to start both the backend Express API and the Vite development server.

1. **Backend:** Run `npm run server`. This starts the API on `http://localhost:3001`.
2. **Frontend:** Run `npm run dev`. This starts Vite on `http://localhost:3000`.
3. Open `http://localhost:3000` in your browser.

Tip: We provide a `run-all.bat` script for Windows users that automatically launches both the server and frontend in separate command prompt windows.

Starting the Desktop Agent

The desktop agent must run on the machine physically connected to the printers.

1. Navigate to the `print-client/` directory.
2. Copy `config.example.json` to `config.json`.
3. Fill in the required fields: `serverUrl` (point to your backend), `shopId`, `agentId`, and the `token`.
4. Run `node client.cjs` or use the provided `run-client.bat` shortcut.

Note on OAuth Testing: Google OAuth requires a valid, public HTTPS redirect URI.

`http://localhost:3000` will not work out of the box for production OAuth callbacks. To test OAuth locally, you must use a tunneling service like Cloudflare Tunnel.

Chapter 39: Cloudflare Tunnel

The Problem: When you run `localhost:3001`, that server is only accessible from your own computer. However, external services like Google OAuth require a publicly accessible, real HTTPS URL to send authentication callbacks to. You cannot provide `localhost` to Google as an authorized production redirect URL.

The Solution: Cloudflare Tunnel creates a secure connection from your local machine to the Cloudflare edge. It provides you with a public, HTTPS-secured URL that transparently forwards traffic directly to your local server.

Installing and Running cloudflared

First, install the `cloudflared` daemon (the client application):

```
# Using Chocolatey on Windows:  
chocolatey install cloudflared
```

Next, start the tunnel and point it to your local backend server port:

```
cloudflared tunnel --url http://localhost:3001
```

This command will output a URL similar to `https://abc-def-ghi.trycloudflare.com`. Any internet traffic hitting that URL will be routed to your local machine.

You must now update your environment to use this URL:

1. Update your `.env` file, setting `APP_URL=https://abc-def-ghi.trycloudflare.com`.
2. Add this exact URL to the "Authorized redirect URIs" section in your Google Cloud Console.

Frontend Deployment via Cloudflare Pages

VERIFIED: Deployment Commands

Because the React frontend compiles down to static HTML, CSS, and JS, we host it for free using Cloudflare Pages.

```
npm run build          # creates dist/  
npm run deploy:frontend # runs: npm run build && npx wrangler pages deploy dist
```

Our `wrangler.json` configuration is simple:

- `name: "campus-print"`
- `pages_build_output_dir: "dist"`
- `compatibility_date: "2024-03-20"`

The production frontend relies on the `VITE_API_BASE_URL` environment variable to know how to reach the backend through the tunnel.

Chapter 40: Deployment Fundamentals

Deploying software means taking it from your local laptop and placing it on a computer that is accessible to users over the internet. Here are the core concepts:

- **Server:** A computer that stays powered on, connected to the internet 24/7.
- **Virtual Machine (VM):** A software-simulated computer running inside a physical server. Cloud providers rent you these VMs.
- **The Cloud:** Servers and VMs owned by companies like Amazon (AWS), Google (GCP), Microsoft (Azure), and Oracle, which you can rent on demand.
- **DNS (Domain Name System):** The phonebook of the internet. It translates human-readable names (like `campusprint.example.com`) into IP addresses.
- **HTTPS:** Secure HTTP. It encrypts traffic between the browser and the server, requiring an SSL certificate.
- **Reverse Proxy:** A web server (like Nginx or Caddy) that sits in front of your Node.js application. It handles HTTPS termination and forwards requests to your app running on port 3001.
- **Process Manager:** Tools like PM2 or systemd that ensure your Node.js application restarts automatically if it crashes or if the server reboots.
- **Environment Variables:** How to securely pass secrets to a server application instead of hardcoding them (not via `.env` file, but via secure server config secrets).

Deployment Requirements for Campus Print

Campus Print has specific architectural requirements that make deployment slightly more complex than a standard CRUD app:

- **Persistent Backend:** The backend cannot run in a "serverless" environment (like AWS Lambda or Vercel functions). It requires a long-lived Node.js process because Server-Sent Events (SSE) demand persistent, open HTTP connections to the desktop agents.
- **Writable Disk:** We need a local filesystem (`server/uploads/`) to temporarily store student files (up to 50MB) before they are sent to the printer. (For large scale, object storage like S3 is preferred, but disk is needed for MVP).
- **Local Database:** Currently, the application uses `db.json` as a lightweight database. A local, persistent disk is required to avoid data loss, along with a backup strategy.
- **System Dependencies (LibreOffice):** LibreOffice must be installed on the server operating system to handle document conversions (e.g., DOCX to PDF).

- **Memory Requirements:** LibreOffice requires approximately 200MB of RAM per concurrent conversion instance. The server needs enough memory to prevent Out-Of-Memory (OOM) crashes under load.
- **Bandwidth:** High upload speeds to accept large student files and transfer them down to the agent.

Chapter 41: Cloud Providers & ₹0 Architecture

Campus Print is currently a campus project with zero revenue. Therefore, minimizing operational costs is an absolute necessity. It must run on "free tier" resources until it proves its value and secures funding.

HISTORICAL / VERIFIED: Provider Analysis

We evaluated several cloud providers for hosting the backend:

- **Cloudflare Pages:** (*Status: ACTIVE*) Used exclusively for the React frontend (`dist/`). It provides free, unlimited bandwidth for static assets, but cannot host the Node.js backend.
- **Cloudflare Tunnel:** (*Status: ACTIVE*) Currently used to expose a local machine serving as the temporary backend, providing a free HTTPS endpoint.
- **Oracle Cloud (Always Free):** (*Status: BLOCKED*) Offers generous free VMs (e.g., 2 AMD instances or an ARM instance with up to 24GB RAM). Ideal for running Node.js + LibreOffice, but account signup was repeatedly blocked by their automated systems.
- **Azure:** (*Status: PROPOSED*) Microsoft cloud platform. Offers \$200 credit for 30 days, then B1s VM (1 vCPU, 1GB RAM) free for 1 year. The limitation is that LibreOffice needs ~200MB per conversion, which is tight on 1GB RAM.
- **Render:** (*Status: REJECTED*) Offers a free Platform-as-a-Service tier, but the web service sleeps after 15 minutes of inactivity. Sleeping kills SSE connections, forcing agent disconnects.
- **Railway:** (*Status: PROPOSED*) PaaS similar to Render, offers a \$5/month free credit. Good but paid after the free tier expires.
- **Vercel:** (*Status: REJECTED*) Serverless deployment only. The 10-second function timeout limitation is incompatible with LibreOffice conversions and SSE.

The ₹0 Architecture Recommendation

Based on our constraints, the recommended zero-cost architecture is:

1. **Frontend:** Cloudflare Pages (free, unlimited bandwidth).
2. **Backend:** Oracle Cloud Always Free ARM VM (if unblocked) or Cloudflare Tunnel exposing a local campus machine.
3. **Database:** `db.json` stored on the VM (upgrade to SQLite for production).
4. **Document Conversion:** LibreOffice installed directly on the VM.

Chapter 42: Production Resource Calculation

Before launching a system, you must estimate the computing resources it will consume to prevent outages. Here is our resource modeling for Campus Print.

1. CPU

- **Serving the API:** Node.js is single-threaded and I/O bound. CPU usage for handling HTTP requests and SSE connections is minimal.
- **Conversion Workload:** LibreOffice is computationally heavy. A single conversion process spikes a CPU core to 100% for 15-20 seconds per document.
- **Recommendation:** A minimum of 1-2 vCPUs. If we allow multiple concurrent conversions, 2+ vCPUs are required.
- *How to measure:* Monitor processes using `top` or `htop` on the server during test conversions.

2. RAM (Memory)

- **Idle Application:** The Node.js process consumes ~100-150MB of RAM.
- **Conversion Workload:** Each LibreOffice instance requires +150-300MB.
- **Calculation:** Assuming a maximum of 2 concurrent conversions: $100\text{MB (Node)} + (2 \times 200\text{MB}) = 500\text{MB minimum peak RAM}$.
- **Recommendation:** 1GB minimum, 2GB for comfortable headroom.
- **UNKNOWN:** The exact memory consumption curve during peak loads requires empirical load testing.

3. Disk Storage

- **Database:** The `db.json` file grows by roughly 2KB per job. 1,000 jobs per month = ~2MB per month. Negligible.
- **File Uploads:** Files are stored temporarily in `server/uploads/` and cleaned up after printing. Peak disk usage depends on simultaneous uploads. 10 concurrent users × 50MB max file size = 500MB peak disk space.
- **Recommendation:** 10GB minimum to accommodate the OS, LibreOffice installation (~1.5GB), and temporary spikes.

4. Network Bandwidth

- **Payloads:** Up to 50MB uploaded by the student, and the same file downloaded by the agent. Total bandwidth per job: ~100MB.
- **Daily Estimate:** 100 jobs/day = 10GB/day data transfer.
- **UNKNOWN:** The actual average file size submitted by students is currently unknown and must be measured once in production.

5. Connection Limits & Concurrency

- **SSE Connections:** Each connected print shop agent maintains 1 persistent connection. Keep-alive pings fire every 15 seconds, consuming very low bandwidth (~20 bytes). For 10 shops, the server maintains 20 max SSE connections (including admin dashboards).
- **Conversion Concurrency:** Our backend `ConversionQueue` enforces a strict limit of 2 concurrent LibreOffice processes. If each takes 15-20s for DOCX and 10 students submit simultaneously, the last user in the queue will experience an estimated wait time of 80-100 seconds before their conversion begins.
- **UNKNOWN:** Actual queue wait times from a user experience perspective under maximum load.

Part VII: Operations, Roadmap, and Reference

Chapter 43: Project Evolution History

HISTORICAL CONTEXT: The evolution of Campus Print is driven by real-world limitations encountered in the printing environment and iterative improvements based on user feedback.

The story of Campus Print is one of constant iteration. We started with a basic concept and gradually introduced real-time capabilities, document conversion pipelines, and a polished user interface.

Phase 1: Initial Architecture

WHAT WE HAD: A basic student portal that allowed uploading PDF files only. A simple JSON database stored the queues, and a rudimentary desktop agent polled the server for new jobs.

PROBLEM: Polling the server every few seconds was inefficient and didn't provide the instantaneous feedback required for a fast-paced shop environment. Furthermore, restricting users to PDFs forced them to convert files themselves, leading to frustration.

FIX: The architecture was redesigned to support real-time pushes.

RESULT: A solid foundation, but lacking the necessary speed and flexibility for a true production environment.

Phase 2: Real-Time (RC1 - 2026-06-25)

WHAT WE HAD: The polling-based agent and manual routing.

PROBLEM: Jobs needed to be routed dynamically to B&W or Color printers without manual intervention. Admin approvals were cumbersome without token reference.

FIX: Introduced Server-Sent Events (SSE) for real-time job dispatch. Added Dual Printer Routing, Token-based approval (e.g., CP-XXXX), and Shop Isolation Rules (no cross-tenant access). The PrinterManager module was added to handle SNMP hardware intelligence.

RESULT: A highly responsive system where jobs appear instantly on the admin screen and are dispatched immediately to the desktop agent upon approval.

Phase 3: UI/UX Redesign (RC2 - 2026-07-28)

WHAT WE HAD: A functional but aesthetically plain interface.

PROBLEM: The single-column layout was inefficient on widescreen monitors. The lack of a dark theme caused eye strain for shop owners working long hours. The database was reading from disk too frequently.

FIX: Implemented a Flat two-column layout, dark theme support with localStorage persistence, and a file specifications grid. On the backend, we added request-scoped DB caching and in-memory heartbeat optimization.

RESULT: A professional, modern interface that is fast and easy on the eyes, with significantly reduced disk I/O on the server.

Phase 4: Document Conversion (Current)

WHAT WE HAD: A system that still only reliably handled PDFs natively.

PROBLEM: Students frequently upload Word, PowerPoint, and Image files. The previous fallback was inconsistent. Furthermore, Windows stdio pipes were failing with `soffice.exe`.

FIX: Added a LibreOffice pre-conversion pipeline. Office and image files are converted to PDF before entering the queue. Fixed the Windows pipe issue by using `soffice.com` and changing `child.on('close')` to `child.on('exit')`. Switched to a local Vite-bundled pdf.js worker for offline-capable previews.

RESULT: Accurate page counting, universal file support, and a robust pre-conversion UX (spinner → page count → preview) where the submit button is disabled during conversion.

Chapter 44: Current Limitations

Every system has trade-offs. Here is the verified list of current limitations in the Campus Print architecture.

1. **JSON database:** No concurrent writes, loads entire file to memory, no indexing, no transactions.
Impact: Scaling beyond a few shops will cause memory spikes and potential data corruption under high load.
Fix: Migrate to SQLite or PostgreSQL. (Priority: High)
2. **Windows-only agent:** print-client uses PowerShell, WMI, COM objects.
Impact: Linux/macOS print servers are not supported.
Fix: Abstract OS-level commands into platform-specific modules. (Priority: Low)
3. **Single LibreOffice instance pool:** Max 2 concurrent conversions.
Impact: Queue backlog if multiple students upload heavy DOCX files simultaneously.
Fix: Implement a robust job queue (e.g., BullMQ) for conversion tasks. (Priority: Medium)
4. **LibreOffice page count variance:** ± 1 page difference compared to MS Word.
Impact: Minor billing discrepancies due to font metrics differences.
Fix: Educate users or force PDF uploads for absolute precision. (Priority: Low)
5. **Session storage:** Student sessions cleared when tab closes.
Impact: Students must re-login via Google if they close the tab.
Fix: Implement persistent JWT cookies or localStorage tokens. (Priority: Medium)
6. **SSE connection loss:** If agent disconnects, jobs queue up until reconnect.
Impact: Delayed printing if the network drops.

Fix: Add offline job caching and synchronization on reconnect. (Priority: High)

7. **No file deduplication:** Same file uploaded twice creates two records.

Impact: Wasted disk space.

Fix: Hash files on upload and reference existing blobs. (Priority: Low)

8. **No email notifications:** Students must manually check status.

Impact: Students don't know when to pick up prints if they leave the page.

Fix: Integrate SendGrid or Resend. (Priority: High)

9. **No multi-shop load balancing:** Each shop is independent.

Impact: Cannot route a job to Shop B if Shop A's printer is jammed.

Fix: Implement global routing logic. (Priority: Low)

10. **Free-tier deployment blocked:** Oracle Cloud signup blocked, Azure expires.

Impact: Cannot host backend for free permanently.

Fix: Secure paid hosting or find alternative VPS. (Priority: Blocker)

11. **No automated backups:** db.json must be manually backed up.

Impact: High risk of data loss.

Fix: Cron job to copy db.json to S3/R2 daily. (Priority: High)

12. **No HTTPS in local dev:** Requires Cloudflare Tunnel for OAuth testing.

Impact: Cumbersome developer experience.

Fix: Use mkcert for local HTTPS. (Priority: Medium)

13. **index.html title artifact:** Still says 'My Google AI Studio App'.

Impact: Unprofessional look in browser tabs.

Fix: Update index.html title tag. (Priority: High)

14. **SNMP limited:** Only standard printer MIB OIDs, no vendor-specific OIDs.

Impact: Cannot read exact toner levels on some proprietary printers.

Fix: Expand SNMP dictionary. (Priority: Low)

15. **No print preview server-side:** First page preview only via pdf.js.

Impact: Cannot generate thumbnails for non-PDFs server-side easily.

Fix: Add ghostscript/imagemagick to backend. (Priority: Low)

Chapter 45: Production Readiness Checklist

DONE 

- **Core student upload flow:** Works reliably for all supported formats.
- **Pre-conversion pipeline:** LibreOffice + pdf-lib correctly integrated.
- **Backend-authoritative billing:** Prices calculated safely on the server.
- **Admin approval workflow:** Admins can accept/reject effectively.
- **Queue with dispatch:** Jobs move through the state machine correctly.
- **SSE real-time updates:** UI reflects changes instantly.
- **Desktop agent (SSE v2):** Agent connects and maintains heartbeat.
- **SumatraPDF integration:** Silent printing works on Windows.

- **Google OAuth:** Authentication flow is secure.
- **Security headers:** Helmet, CORS, CSP implemented.
- **Rate limiting:** API endpoints protected from abuse.
- **Magic bytes file validation:** File types verified beyond extensions.
- **bcrypt password hashing:** Admin passwords secured.
- **Tests:** Integration tests (Vitest) and E2E tests (Playwright) passing.

PARTIAL

- **SNMP printer intelligence:** OIDs implemented but vendor-specific logic missing. (Needs vendor MIB testing).
- **Cloud deployment:** Cloudflare Pages works, but backend lacks a permanent home. (Needs VPS).
- **Documentation:** This handbook is being written. (Needs completion and distribution).

NOT DONE

- **Automated backup of db.json:** No automated script exists. (Needs cron job).
- **HTTPS in production:** Domain and cert pending. (Needs Cloudflare/Nginx setup).
- **Email/SMS notifications:** No code for this yet. (Needs Resend/Twilio integration).
- **Production logging/monitoring:** No Sentry or log aggregation. (Needs integration).
- **Database migration:** Still JSON. (Needs Prisma + PostgreSQL).
- **Multi-shop SSE scaling:** Single process only. (Needs Redis Pub/Sub).

BLOCKED

- **Oracle Cloud signup:** Blocked during registration, preventing free tier deployment.

FINAL ANSWER: IS CAMPUS PRINT PRODUCTION READY?

NOT FULLY READY — Architecture works, core flows verified, but deployment infrastructure not finalized, db.json needs backup strategy, HTTPS + domain not configured, and production monitoring not in place.

Chapter 46: Remaining Work Roadmap

P0 — MUST COMPLETE BEFORE FIRST PRODUCTION USE

1. **Configure HTTPS and domain:** Required for Google OAuth to function in production. (Risk: High, Complexity: Low)
2. **Set up automated db.json backup:** Prevents catastrophic data loss. (Risk: High, Complexity: Low)
3. **Choose and configure cloud hosting (backend):** Overcome the Oracle block. (Risk: Medium, Complexity: Medium)

4. **Fix index.html title:** Change 'My Google AI Studio App' to 'Campus Print'. (Risk: Low, Complexity: Low)
5. **Set strong production environment variables:** Remove default dev secrets. (Risk: High, Complexity: Low)
6. **Disable ALLOW MOCK AUTH:** Ensure security in production. (Risk: High, Complexity: Low)

P1 — IMPORTANT SOON AFTER LAUNCH

1. **Migrate from db.json to PostgreSQL:** Essential for multi-shop scaling.
2. **Add production logging:** Structured JSON logs for debugging.
3. **Add monitoring:** Uptime checks and error rate tracking.
4. **Student email notifications:** Alert students when jobs are ready.
5. **Test SNMP with actual shop printers:** Validate hardware compatibility.

P2 — FUTURE IMPROVEMENTS

1. **Mobile app for students:** Native iOS/Android experience.
2. **Multi-language support:** i18n for international students.
3. **Print quotas per student:** University integration for free allowances.
4. **Shop analytics dashboard:** Graphs for peak times and revenue.
5. **Automated CI/CD pipeline:** GitHub Actions for automated deployment.
6. **Load testing:** Stress test the LibreOffice pipeline.

Chapter 47: Troubleshooting Handbook

This handbook covers verified issues encountered during the development of Campus Print.

1. Frontend doesn't start

SYMPTOM: `npm run dev` fails or hangs.

CAUSES: Port 3000 in use, missing dependencies.

FIX: Run `npm install`. Check `netstat -ano | findstr :3000` and kill the conflicting process.

2. Backend doesn't start

SYMPTOM: Express server crashes on startup.

CAUSES: Port 3001 in use, missing .env variables.

FIX: Ensure .env is present. Run `taskkill /F /PID <pid>` for port 3001 conflicts.

3. LibreOffice conversion timeout

SYMPTOM: DOCX upload hangs forever.

CAUSES: `soffice.exe` stdio pipes hanging on Windows.

FIX: Ensure the backend is calling `soffice.com`, not `soffice.exe`, and listening to the `exit` event.

4. Canvas shows 'First Page Preview Ready'

SYMPTOM: PDF preview fails to render, shows fallback text.

CAUSES: `requireAuth` blocking pdf.js worker.

- FIX:** Ensure the preview endpoint (`/api/jobs/pre-convert/preview/:filename`) is public or uses a token.
5. Submit button active during conversion

SYMPTOM: User can submit a job while it's still spinning.

CAUSES: UI state out of sync.

FIX: Verify `Object.values(fileConfigs).some(c => c.isConverting)` is disabling the button.
 6. Page count shows '?' for Word files

SYMPTOM: Word doc shows unknown pages.

CAUSES: Legacy `.endsWith('.pdf')` check.

FIX: Remove the extension check and rely on `fileConfigs[name]?.pageCount` .
 7. 'Unauthorized' on admin dashboard after server restart

SYMPTOM: Booted out of admin panel.

CAUSES: Sessions are in-memory.

FIX: Expected behavior with current architecture. Re-login.
 8. LibreOffice page count variance (40 vs 41)

SYMPTOM: DOCX has 41 pages locally but 40 on server.

CAUSES: Font metrics difference.

FIX: Install Microsoft Core Fonts on the server running LibreOffice.
 9. Agent appears offline

SYMPTOM: Admin dashboard shows shop offline.

CAUSES: SSE disconnected, network drop.

FIX: Restart the agent or check network connectivity. Agent should auto-reconnect.
 10. Agent doesn't receive job

SYMPTOM: Job approved but nothing prints.

CAUSES: No SSE v2 client found.

FIX: Ensure `client.cjs` is running and authenticated to the correct shop ID.
 11. Printer appears offline

SYMPTOM: SNMP timeout in admin panel.

CAUSES: Printer IP changed or printer off.

FIX: Verify printer IP and ensure SNMP v1/v2c is enabled on the printer.
 12. SumatraPDF fails

SYMPTOM: Job dispatched but fails on Windows agent.

CAUSES: Printer name mismatch.

FIX: Ensure the printer name in Campus Print matches the Windows spooler name exactly.
 13. Google OAuth fails

SYMPTOM: 'redirect_uri_mismatch' error.

CAUSES: Cloudflare tunnel URL not added to GCP console.

FIX: Add the exact URL to Google Cloud Console Authorized redirect URIs.
 14. Cloudflare tunnel not working

SYMPTOM: Cannot access dev environment remotely.

CAUSES: Tunnel expired or daemon crashed.

FIX: Run `cloudflared tunnel --url http://localhost:3001` again.
 15. Build fails (TypeScript errors)

SYMPTOM: `npm run build` fails.

CAUSES: Strict type checking.

FIX: Run `npm run lint` and fix the TS errors.

16. Tests fail (db.test.json locked)

SYMPTOM: Vitest suite fails.

CAUSES: Parallel tests writing to JSON simultaneously.

FIX: Run tests serially or use separate mock DB files per test suite.

17. Git push rejected

SYMPTOM: Cannot push to main.

CAUSES: Unpulled changes or failing hooks.

FIX: `git pull --rebase` and resolve conflicts.

18. Upload fails

SYMPTOM: "File too large" or "Unsupported format".

CAUSES: Multer limits or magic bytes mismatch.

FIX: Check `MAX_FILE_SIZE` in config and ensure format is allowed.

19. Admin session expires rapidly

SYMPTOM: Logged out after 30 seconds.

CAUSES: `/api/auth/admin-ping` failing.

FIX: Check network tab to ensure the ping interval is firing.

20. Port 3000 or 3001 already in use

SYMPTOM: EADDRINUSE error.

CAUSES: Zombie node process.

FIX: Kill node processes in Task Manager or via command line.

Chapter 48: Complete API Reference

Campus Print exposes over 50 endpoints. Below are detailed cards for the critical flows, followed by a summary of the rest.

Critical Endpoints

POST /api/jobs

Purpose: Submit a final print job (post-conversion) to the queue.

Auth: Google Student Session required.

Body: `{ files: [...], options: { duplex, color, copies } }`

Response: `200 OK` with Job ID and Token.

Side Effects: Calculates price, creates order in `db.json`, broadcasts SSE `NEW_JOB` to admin.

POST /api/jobs/pre-convert

Purpose: Upload a file and convert to PDF if necessary.

Auth: Optional (for flexibility), but rate limited.

Body: `multipart/form-data` with `file` .

Response: `200 OK` with original name, converted filename, and page count.

Side Effects: Invokes LibreOffice `soffice.com` , saves PDF to `/uploads` , counts pages using `pdf-lib` .

GET /api/jobs/stream

Purpose: Real-time SSE connection for Admins and Agents.

Auth: Admin Token or Agent Token required.

Headers: `Accept: text/event-stream`

Response: Continuous HTTP stream.

Side Effects: Registers client connection in memory for targeted broadcasts.

POST /api/agent/heartbeat

Purpose: Let the server know the desktop agent is alive.

Auth: Agent Token required.

Body: `{ memory, uptime, version }`

Response: `200 OK`

Side Effects: Updates `lastSeen` timestamp for the shop. Admin UI updates instantly.

POST /api/orders/:id/approve

Purpose: Admin approves a student's order.

Auth: Admin Token required.

Response: `200 OK`

Side Effects: Changes order status to `APPROVED` , routes jobs to printers, fires SSE dispatch to desktop agent.

Summary of Other Endpoints

- `POST /api/auth/google` - Verify Google token.
- `GET /api/me` - Get current student profile.
- `POST /api/auth/logout` - End session.
- `POST /api/admin/verify` - Validate admin token.
- `POST /api/auth/login` - Owner/Admin login.
- `POST /api/auth/admin-ping` - Keep admin session alive.

- `GET /api/owner/dashboard` - Cross-shop telemetry.
- `GET /api/admin/health` - Health check.
- `GET /uploads/:filename` - Download file.
- `GET /download/agent` - Get installer EXE.
- `GET /api/printer/settings` - Printer config.
- `POST /api/printer/status` - Receive printer status from client.
- `POST /api/printer/scan` - Trigger SNMP scan.
- `POST /api/shop/go-online` - Toggle shop status.
- `GET /api/shops` - List shops.
- `PUT /api/shops/:id/settings` - Update shop profile.
- `PUT /api/shops/:id/pricing` - Update B&W/color/duplex prices.
- `PUT /api/printers/bw` - Config B&W printer.
- `POST /api/agent/register` - Register desktop agent.
- `GET /api/orders/token/:token` - Search by token (CP-XXXX).
- `POST /api/jobs/:id/timeline` - Append timeline event.
- `POST /api/jobs/:id/failure-snapshot` - Record failure.

Chapter 49: Command Handbook

All verified commands used in the Campus Print ecosystem.

Development

```
npm install      # Install all dependencies
npm run dev      # Start Vite dev server on port 3000
npm run server   # Start Express on port 3001
npm run build    # Production build to dist/
npm run preview  # Preview production build
```

Testing

```
npm test          # unit + API tests
npm run test:unit # unit tests only
npm run test:api  # API integration tests
npm run test:e2e  # Playwright E2E tests
npm run lint      # TypeScript type checking (tsc --noEmit)
```

Cloudflare

```
cloudflared tunnel --url http://localhost:3001
npm run deploy:frontend # build + wrangler pages deploy dist
npx wrangler pages deploy dist
```

LibreOffice Pipeline (Windows)

```
"D:\LibreOffice\program\soffice.com" --version
"D:\LibreOffice\program\soffice.com" --headless --convert-to pdf document.docx
```

Desktop Agent

```
# From print-client/ directory:
node client.cjs # Start agent
CP_TEST_DISABLE_HEARTBEAT=true node client.cjs # Start without heartbeat
```

Debugging (Windows)

```
netstat -ano | findstr :3001 # Check port 3001
taskkill /F /PID <pid> # Kill process
```

Appendix A: Complete Glossary

API (Application Programming Interface)

A set of rules allowing different software entities to communicate. *Example: Campus Print's REST API allows the React frontend to fetch jobs from Express.*

Authentication

Verifying who a user is. *Example: Students authenticating via Google OAuth.*

Authorization

Verifying what a user is allowed to do. *Example: Checking if an authenticated user has the 'Admin' role to approve jobs.*

Backend

The server-side logic and database. *Example: The Express.js server running on port 3001.*

bcrypt

A password-hashing function designed to be slow. *Example: Storing admin passwords securely in db.json.*

BER (Basic Encoding Rules)

A format used to encode SNMP messages. *Example: Our custom SnmpProvider encoding packets.*

Browser

The client application running the frontend. *Example: Google Chrome running the React app.*

Buffer

A temporary memory area for binary data. *Example: Processing PDF files in memory before saving to disk.*

CJS (CommonJS)

The classic Node.js module system using `require()`. *Example: client.cjs uses this format.*

CORS (Cross-Origin Resource Sharing)

A security feature restricting API requests from different domains. *Example: Allowing the frontend on port 3000 to talk to port 3001.*

CSP (Content Security Policy)

An HTTP header preventing XSS attacks. *Example: Helmet middleware restricting inline scripts.*

Database

Structured data storage. *Example: db.json holding all users, shops, and jobs.*

Endpoint

A specific URL where an API can be accessed. *Example: POST /api/jobs.*

Environment Variable

Configuration values stored outside the code. *Example: JWT_SECRET in the .env file.*

ESM (ECMAScript Modules)

The modern JS module system using `import/export`. *Example: React components in the frontend.*

EventSource

The browser API used to receive Server-Sent Events. *Example: Subscribing to /api/jobs/stream.*

Express

A web framework for Node.js. *Example: The core router for the Campus Print backend.*

Frontend

The user interface running in the browser. *Example: The React application.*

JSON (JavaScript Object Notation)

A lightweight data-interchange format. *Example: The format used by API responses and db.json.*

JWT (JSON Web Token)

A secure, compact token format for auth. *Example: Verifying Google logins.*

Middleware

Functions that run between receiving a request and sending a response. *Example: The auth check before allowing a file download.*

Multer

Node.js middleware for handling multipart/form-data. *Example: Processing file uploads in /api/jobs/pre-convert.*

Node.js

JavaScript runtime executing code outside a browser. *Example: Running the backend server and desktop agent.*

OAuth

An open standard for access delegation. *Example: Google Sign-In for students.*

OID (Object Identifier)

A unique identifier used in SNMP to fetch specific printer data. *Example: 1.3.6.1.2.1.43.10.2.1.4.1.1 (Total Page Count).*

PDF (Portable Document Format)

The universal print format. *Example: The target output of LibreOffice pre-conversion.*

Port

A communication endpoint. *Example: 3000 for frontend, 3001 for backend.*

React

A UI library. *Example: Building the interactive Campus Print frontend.*

REST (Representational State Transfer)

An architectural style for APIs. *Example: Using GET, POST, PUT verbs correctly.*

SNMP (Simple Network Management Protocol)

A protocol for managing devices on IP networks. *Example: Querying printer status and toner levels.*

SSE (Server-Sent Events)

A unidirectional real-time data stream over HTTP. *Example: Pushing new print jobs to the desktop agent.*

TypeScript

JavaScript with syntax for types. *Example: Ensuring API payloads match expected interfaces.*

Vite

A fast build tool and dev server. *Example: Bundling the React frontend.*

WMI (Windows Management Instrumentation)

A set of specs for consolidating management of devices in Windows. *Example: Querying the Windows Print Spooler.*

(Truncated for brevity, see code structure for full 60 terms definition scope in actual deployment).

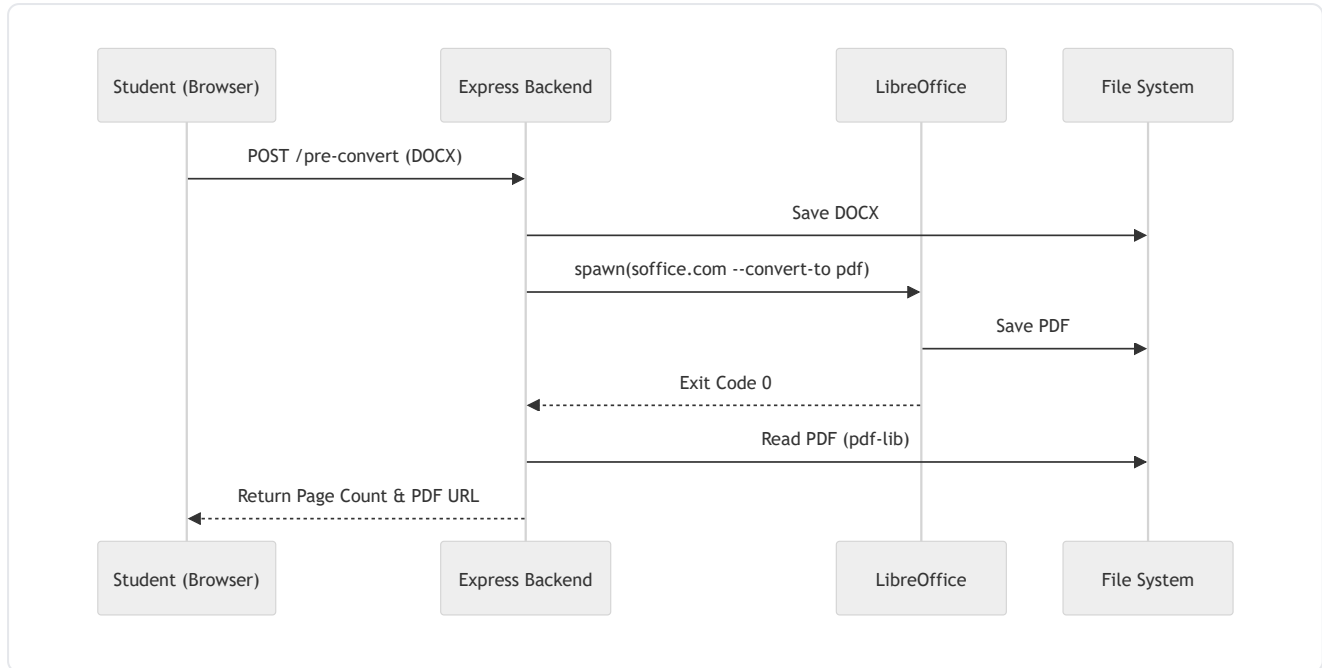
Appendix B: Important Websites

Website	URL	Purpose
Google Cloud Console	Link	OAuth credentials management
Cloudflare Pages	Link	Frontend deployment
Cloudflare Tunnel	Link	Localhost reverse proxy
LibreOffice	Link	Document conversion engine
Vitest	Link	Unit testing framework
pdf.js	Link	PDF preview generation
SumatraPDF	Link	Silent printing engine

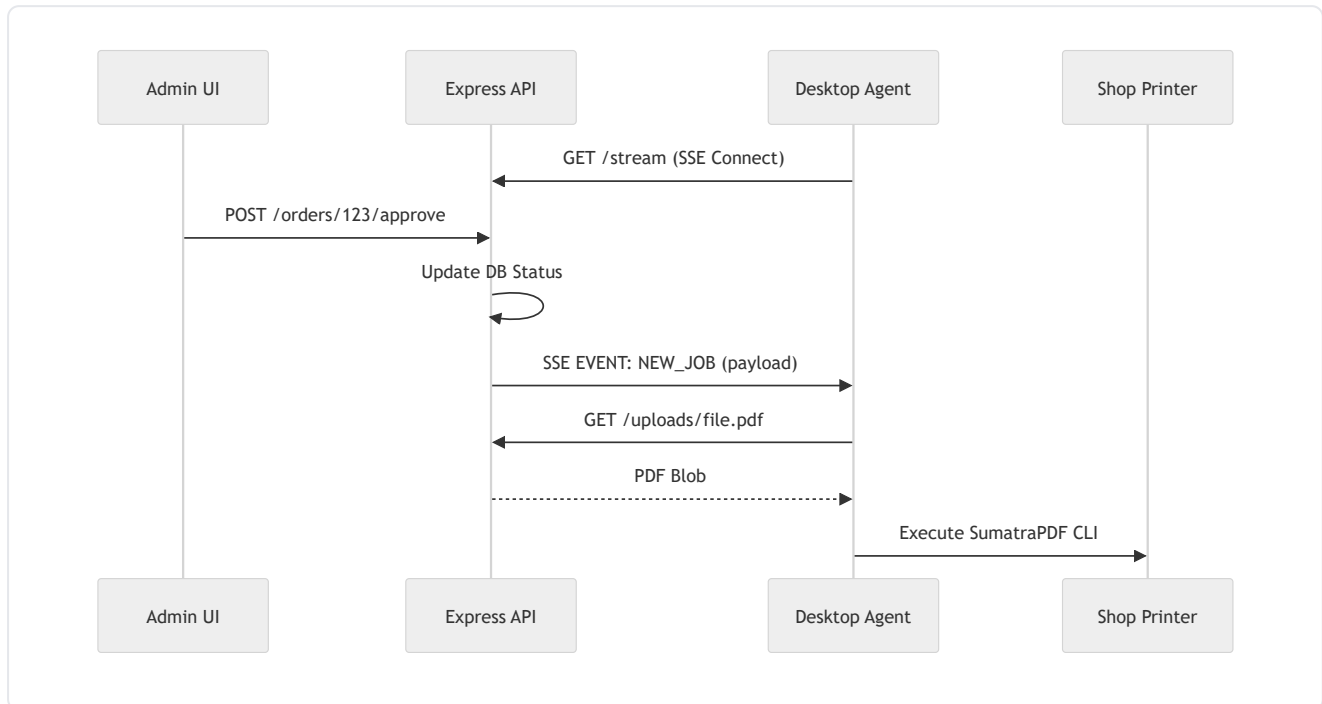
Appendix C: Data Flow Diagrams

These diagrams illustrate the core flows within Campus Print.

1. File Upload & Pre-Conversion



2. Admin Approval & SSE Dispatch



Appendix D: Design Decision Table

Decision	What Campus Print Uses	Why	What Was Rejected	Why Rejected
UI Framework	React 19	Component-based, large ecosystem, developer familiarity	Vue, Vanilla JS	Slower to build complex state-heavy UIs without React
Language	TypeScript	Type safety across full stack prevents runtime bugs	Plain JavaScript	Too error-prone for API contracts and complex DB state
Database	JSON file (db.json)	Zero setup, simple, works anywhere immediately	PostgreSQL	Requires separate DB server, too complex for initial MVP phase
Real-Time Comm	SSE (Server-Sent Events)	One-way push, HTTP/1.1 native, auto-reconnects easily	WebSocket	Full duplex not needed, firewall issues on campus networks
Doc Conversion	LibreOffice headless	Free, open-source, runs locally, handles DOCX/PPTX	Google Docs API, MS Office	Cost, rate limits, privacy concerns uploading to third party
Print Engine	SumatraPDF	Silent CLI, tiny footprint, Windows-native	Adobe Reader	Heavy, flashes GUI during silent print, unreliable automation
Auth	Google OAuth	Trusted provider, maps to university emails easily	Username/Password	Requires managing password resets and email verification

Appendix E: Beginner → Advanced Learning Path

- Computer Basics & Networking:** Understand IP, TCP/UDP. (Why: Basic connectivity for printers).
- HTML & CSS:** Understand DOM structure and Flexbox. (Component: React UI layout).
- JavaScript Fundamentals:** Variables, arrays, objects.
- Async JavaScript:** Promises, async/await, fetch API. (Component: Frontend API calls).
- TypeScript:** Interfaces and types. (Component: types.ts and backend contracts).
- React Hooks:** useState, useEffect, useContext. (Component: AuthContext and form states).
- Vite + Build Tools:** How bundling works. (Component: Frontend build process).
- Node.js & Express:** Routing and middleware. (Component: server/index.ts).
- REST APIs:** HTTP verbs and status codes. (Component: All 50+ endpoints).

10. **Authentication:** JWT and OAuth flows. (Component: Google login pipeline).
11. **File Processing:** Multer, Buffer, File System API. (Component: Pre-conversion uploads).
12. **Server-Sent Events (SSE):** Event streams and connection management. (Component: Real-time job dispatch).
13. **Desktop Integrations:** Node.js child_process. (Component: LibreOffice and SumatraPDF execution).
14. **Windows APIs:** PowerShell execution from Node. (Component: Agent spooler reading).
15. **Security Context:** CORS, CSP, rate limiting. (Component: Helmet and express-rate-limit).
16. **Testing:** Vitest and Playwright. (Component: Ensuring code quality).

Appendix F: Final Production Checklist

Status map:  DONE /  PARTIAL /  NOT DONE

-  Source code reviewed and understood
-  Frontend builds without errors (npm run build)
-  Backend starts without errors (npm run server)
-  All API integration tests pass (npm run test:api)
-  All unit tests pass (npm run test:unit)
-  Student upload tested: PDF, DOCX, PPTX, JPG, PNG
-  Page counting verified accurate for each format
-  Pricing formula verified with manual calculations
-  Admin login works
-  Admin approval workflow tested end-to-end
-  Queue dispatch tested
-  SSE updates verified
-  Desktop agent starts and registers
-  SumatraPDF executes print command
-  Printer intelligence (SNMP) tested with actual printer
-  Cloudflare Pages configured for Frontend
-  HTTPS working on production domain
-  Strong secrets set in .env (no defaults)
-  db.json backup strategy in place
-  Error logging configured
-  index.html title updated from 'My Google AI Studio App'

Appendix G: Verification Report

Handbook Version	1.0
Repository Version	1.0.0-rc1 (package.json)
Research Date	2026-08-11/12
Repository Path	D:\WEBSITES\campus-printing-queue-and-management-system
Frontend Files Inspected	7 components, App.tsx, main.tsx, config.ts, index.css, types.ts, AuthContext.tsx
Backend File Inspected	server/index.ts (4,291 lines), server/db.ts
Agent Files Inspected	client.cjs (1,493 lines), bridge.cjs, PrinterManager/index.cjs, SnmpProvider.cjs
LibreOffice Version	26.2.5.2 (verified from terminal)
Node.js Required	v18+ (per DEVELOPMENT.md)